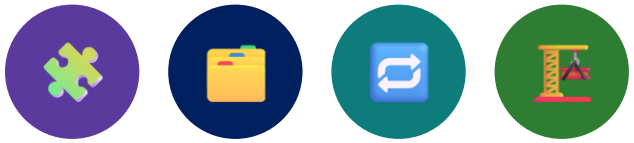
"A component is not a file -- it is a contract between a template and the class behind it."

MODULE 33

Angular Component Architecture

Learn how Angular applications are structured, bootstrapped, and composed from standalone components.



TEKsystems Global Services

WORKFORCE DEVELOPMENT

MODULE 33 OVERVIEW

How Angular applications are built from typed, composable components.

You are joining the crm-ui team: a new Angular front end that will consume the CRM backend built earlier in this bootcamp.

DURATION & FOCUS

4 Hours Theory

Angular architecture, the CLI, component anatomy, templates, binding, inputs/outputs, lifecycle, and folder structure.

Hands-On Focus

Concept walkthroughs and code patterns you will apply directly when crm-ui labs open later in the course.

Scenario

Stand up crm-ui as a standalone-component Angular app that will eventually list and edit CRM customers.

MODULE ARC

Structure First

A workspace, a src layout, and a bootstrap path -- the same shape in every Angular app you will ever open.

Components Compose

Every screen is a tree of small, typed components talking through inputs and outputs.

Standalone by Default

Modern Angular components declare their own imports -- no NgModule required to get started.

KEY TAKEAWAY Angular apps are trees of standalone components; get the CLI, the src layout, and the component anatomy solid before anything else.

WHY COMPONENT ARCHITECTURE MATTERS

A messy component tree costs more than a messy service layer -- it costs every screen.

One Job Per Component

A component that renders a list, fetches data, and formats currency is three components wearing a trenchcoat.

Reuse Without Copy-Paste

A well-scoped CustomerCard component gets used on the list, detail, and search-results screens without duplication.

Testable in Isolation

A component with clear inputs and outputs can be tested and Storybook-previewed without booting the whole app.

KEY TAKEAWAY Component architecture is a design discipline, not a folder convention -- boundaries drawn well here save every screen built after this module.

Why Component Architecture Matters

Angular applications grow in complexity as features, users and business needs expand. A well-designed component architecture provides the structure and consistency needed to manage this complexity effectively.

BENEFITS OF A STRONG COMPONENT ARCHITECTURE

MODULAR AND REUSABLE
Build components that can be reused across the application, reducing duplication and development time.

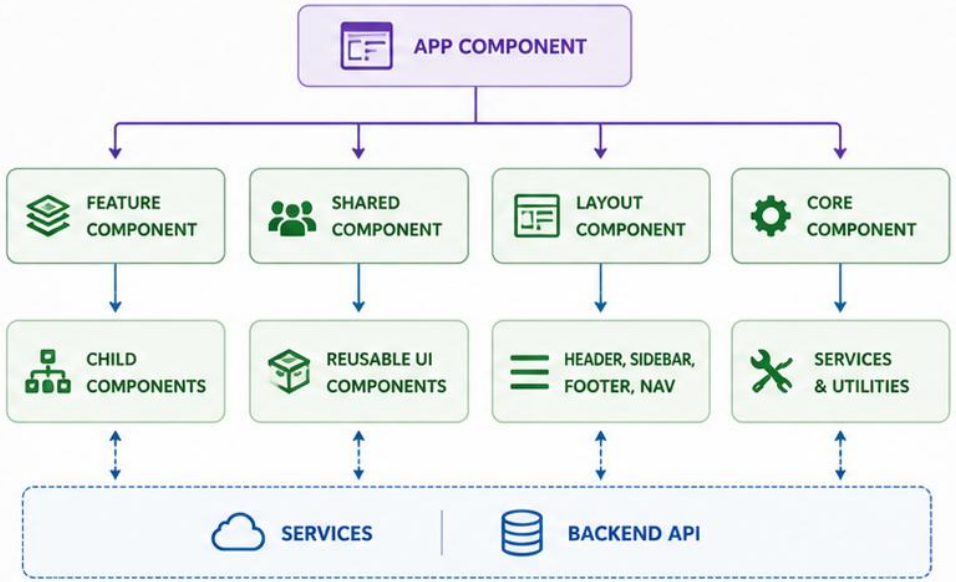
MAINTAINABLE AND SCALABLE
A well-structured architecture makes the codebase easier to maintain and enables the application to scale smoothly.

IMPROVED DEVELOPMENT VELOCITY
Clear component boundaries and communication patterns help teams deliver features faster and with confidence.

BETTER TESTABILITY
Isolated components are easier to unit test, leading to higher quality and fewer regressions.

CONSISTENT USER EXPERIENCE
Reusable components ensure a consistent look and feel across the application.

APPLICATION STRUCTURE WITH COMPONENT ARCHITECTURE



BUSINESS OUTCOMES

- Faster Time to Market
- Higher Developer Productivity
- Lower Maintenance Cost
- Better Scalability for Growth
- Improved User Satisfaction

MODULE LEARNING OBJECTIVES

By the end of this module you can explain and apply every layer of an Angular component.

Objective	Description
Explain Angular's Shape	Describe how a workspace, the CLI, and bootstrap fit together to start crm-ui.
Build Standalone Components	Write a @Component with selector, template, and imports -- no NgModule required.
Bind Data Both Ways	Use interpolation, property, event, and two-way binding correctly and know when each applies.
Communicate Parent to Child	Wire @Input()/input() and @Output()/output() so components talk without tight coupling.
Organize at Scale	Apply feature-based folders, shared components, and DI so crm-ui stays maintainable as it grows.

KEY TAKEAWAY Every objective on this slide maps to a concrete artifact you will produce in crm-ui -- not just vocabulary.

LEARNING OBJECTIVES

By the end of this module, you will be able to:

UNDERSTAND COMPONENT FUNDAMENTALS
Explain what Angular components are and how they form the building blocks of an application.

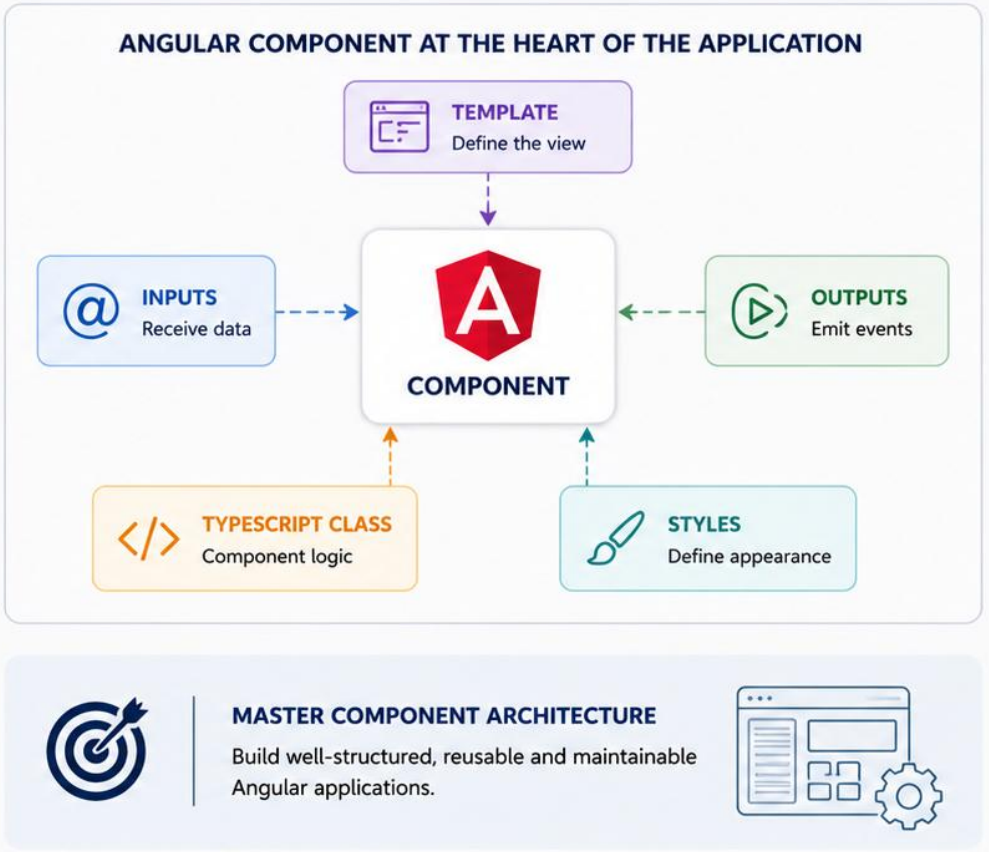
BUILD AND STRUCTURE COMPONENTS
Create components and understand their anatomy, templates, styles, metadata, and TypeScript class.

MANAGE COMPONENT INTERACTION
Use **@Input** and **@Output** to pass data and events between parent and child components.

COMPOSE REUSABLE UI
Design reusable and modular components that promote consistency and reduce duplication.

APPLY ARCHITECTURAL BEST PRACTICES
Organize components using feature-based structure and follow enterprise Angular architecture patterns.

LEVERAGE COMPONENT LIFECYCLE
Understand key lifecycle hooks and how they influence component behavior.



MASTER COMPONENT ARCHITECTURE
Build well-structured, reusable and maintainable Angular applications.

ANGULAR OVERVIEW

A TypeScript-first, component-based framework for building single-page applications.

TypeScript Native

Angular is written in and designed for TypeScript -- types flow from templates through components to services.

Component-Based

Every UI unit, from a button to a full page, is a component with its own template, styles, and logic.

Batteries Included

Routing, forms, HTTP client, and dependency injection ship as first-party Angular packages, not third-party choices.

KEY TAKEAWAY Angular is an opinionated, TypeScript-native platform -- the framework makes more decisions for you than React does, on purpose.

ANGULAR OVERVIEW

Angular is a modern, open-source web application framework developed and maintained by Google.
It is used to build dynamic, scalable, and high-performance single-page applications.

ANGULAR AT A GLANCE

ANGULAR

- **Developed by:** Google
- **Language:** TypeScript
- **Architecture:** Component-Based
- **Data Binding:** One-Way and Two-Way
- **Rendering:** Client-Side (with SSR support)
- **Testing:** Built-in testing tools (Jasmine, Karma)
- **CLI:** Angular CLI for rapid development
- **Latest Version:** Angular 17+ (as of 2024)

KEY HIGHLIGHTS

Component-Based Framework
Build UIs using reusable, encapsulated components.

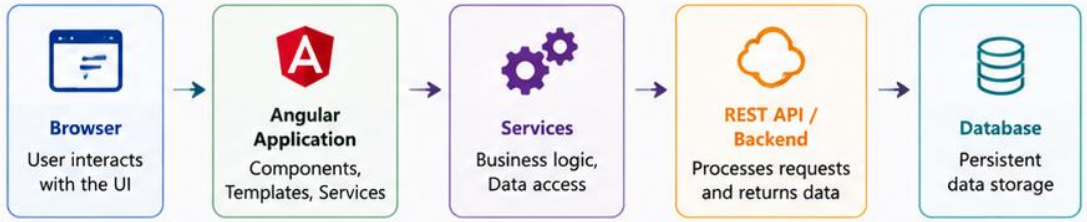
Built with TypeScript
Type-safe, object-oriented, and powerful JavaScript superset.

High Performance
Optimized change detection and code generation.

Cross-Platform
Build web, mobile (Ionic), desktop (Electron) and more.

Robust Ecosystem
Rich set of tools, libraries and community support.

ANGULAR ARCHITECTURE OVERVIEW



Why Angular?
Angular helps teams build maintainable, testable and scalable applications by promoting component reusability, strong structure and modern development practices.

ANGULAR APPLICATION ARCHITECTURE

Four building blocks combine to form every Angular application, from a todo app to crm-ui.

Component Tree

One root component renders a tree of child components -- the entire UI is that tree.

Router

Maps URL paths to components, so /customers and /customers/42 render different screens.

Services + DI

Shared logic (HTTP calls, state, formatting) lives in injectable services, not components.

Reactivity Layer

Signals and RxJS Observables drive what re-renders when data changes -- covered fully in Module 34.

KEY TAKEAWAY A component tree, a router, injectable services, and a reactivity layer -- learn to place new code in the right one of these four.

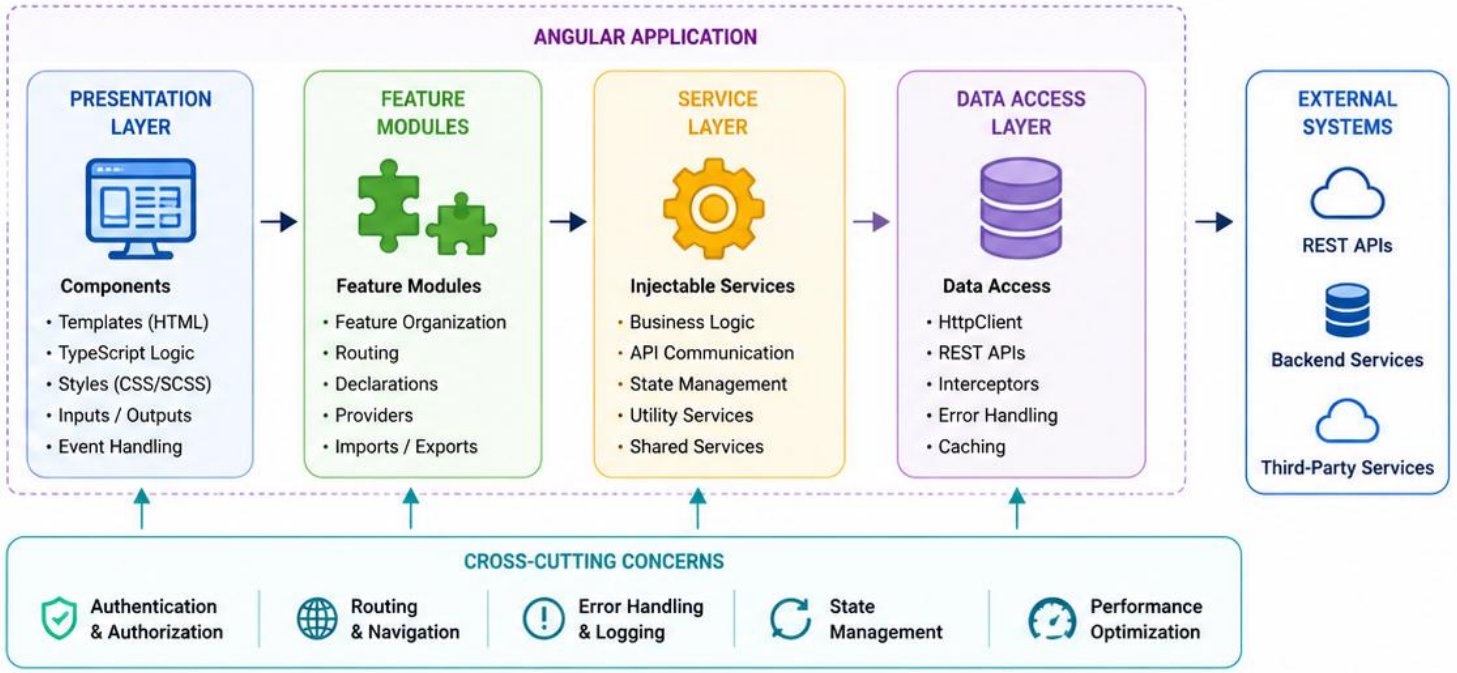
Angular Application Architecture

Angular follows a component-based, modular, and service-oriented architecture designed for scalable, maintainable enterprise applications.

ARCHITECTURAL PRINCIPLES

- **Component-Based UI**
  UI is built using reusable components with their own template, logic, and styles.
- **Modular Structure**
  Features are organized into modules for scalability and separation of concerns.
- **Service Layer**
  Business logic, data access, and cross-cutting concerns are handled by injectable services.
- **Dependency Injection**
  Angular's DI system promotes loose coupling and testability.
- **Reactive and Declarative**
  UI updates automatically with data changes using Angular's reactivity model.

HIGH-LEVEL ARCHITECTURE OVERVIEW



KEY BENEFITS

- Scalable Architecture
- Maintainable Codebase
- Testable Components
- High Performance
- Enterprise Ready

Angular's architecture promotes separation of concerns, reusability, and testability — helping teams build robust and scalable enterprise applications.

TYPESCRIPT IN ANGULAR

Angular is not 'usable with' TypeScript -- it is written assuming TypeScript from the ground up.

Decorators

@Component, @Injectable, and @Input are TypeScript decorators -- metadata attached directly to classes and members.

Static Typing

Component properties, service methods, and template expressions are all type-checked at compile time.

Interfaces & Generics

Shape API responses with interfaces (e.g. Customer) so a typo in a field name fails the build, not the demo.

KEY TAKEAWAY TypeScript's decorators and static types are how Angular attaches metadata to classes and catches template errors before runtime.

TYPESCRIPT IN ANGULAR

Angular is built with TypeScript, a strongly typed superset of JavaScript that adds type safety, modern features, and powerful tooling.

WHY TYPESCRIPT?

Type Safety
Catch errors at compile time instead of runtime.

Better Tooling
IntelliSense, autocompletion, refactoring, and navigation.

Modern JavaScript
Use latest ECMAScript features with backward compatibility.

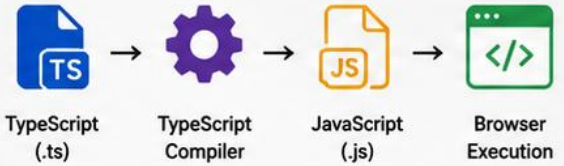
Scalability
Improves maintainability and enables large-scale applications.

Enhanced Readability
Interfaces, types, and clear contracts make code easier to understand.

TYPESCRIPT KEY FEATURES

- Static Typing
- Interfaces & Types
- Classes & OOP
- Arrow Functions & Lambdas
- Modules & ES6+ Support
- Generics

TYPESCRIPT COMPILATION FLOW



EXAMPLE: TYPESCRIPT IN ANGULAR COMPONENT

```typescript
import { Component, Input, OnInit } from '@angular/core';

interface User {
  id: number;
  name: string;
  email: string;
  isActive?: boolean; // optional property
}

@Component({
  selector: 'app-user-card',
  templateUrl: './user-card.component.html'
})
export class UserCardComponent implements OnInit {
  @Input() user!: User;
```

user-card.component.ts

TYPESCRIPT vs JAVASCRIPT

TypeScript	JavaScript
Statically Typed	Dynamically Typed
Compile-time Checking	Runtime Errors
Interfaces & Types	No Built-in Types
Better IDE Support	Basic IDE Support
Easier Refactoring	Harder Refactoring

Key Takeaway: TypeScript brings structure, safety, and productivity to Angular development, enabling teams to build robust and maintainable enterprise applications.

ANGULAR CLI

One command-line tool scaffolds, serves, builds, and tests every Angular project consistently.

Command	What It Does
ng new <name>	Scaffolds a new workspace with a standalone root component and default tooling.
ng generate component <name>	Creates a new standalone component with its own template and class files.
ng serve	Builds the app and serves it locally with live reload, typically at localhost:4200.
ng build	Produces an optimized, production-ready bundle in the dist/ folder.
ng test	Runs the project's unit tests (Karma/Jasmine by default, or Jest in newer setups).

KEY TAKEAWAY The Angular CLI is the one true way to scaffold, run, and build an Angular app -- learn the five commands on this table and you can operate any Angular project.

ANGULAR CLI

Angular CLI (Command Line Interface) is a powerful tool that helps developers create, build, test, and maintain Angular applications efficiently.

WHAT IS ANGULAR CLI?

A command-line tool that automates development tasks so you can focus on building great applications.

KEY BENEFITS

- **Improves Productivity**
  Automates repetitive tasks
- **Consistent Project Structure**
  Follows Angular best practices
- **Built-in Tooling**
  Compile, test, lint, and more
- **Extensible**
  Supports schematics and plugins

COMMON ANGULAR CLI COMMANDS

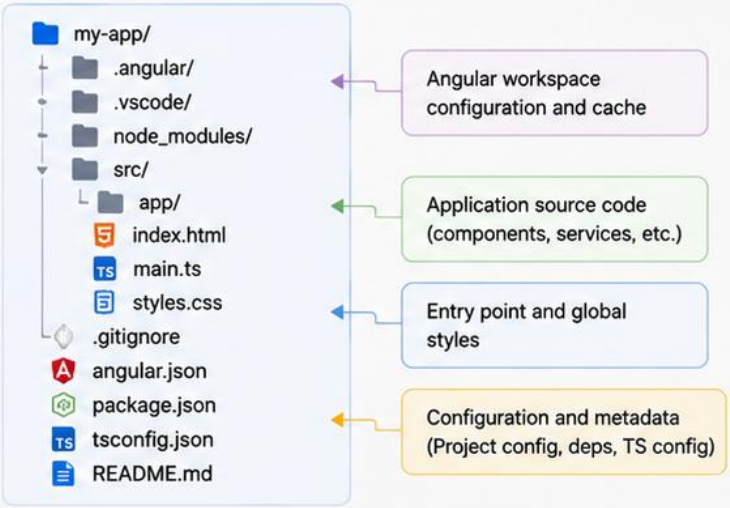
Command	Description
`ng new <project>`	Creates a new Angular application
`ng generate component <name>`	Generates a new component
`ng generate service <name>`	Generates a new service
`ng build`	Builds the application
`ng serve`	Builds and serves the app locally
`ng test`	Runs unit tests
`ng lint`	Runs linting checks
`ng e2e`	Runs end-to-end tests
`ng add <package>`	Adds a third-party package

EXAMPLE: RUNNING THE APPLICATION

```
$ ng new my-app
CREATE my-app/README.md (1062 bytes)
CREATE my-app/angular.json (2711 bytes)
...
$ cd my-app
$ ng serve
✓ Browser application bundle generation complete.
✓ Initial Chunk Files | Names     | Size
        main.js       | main      | 123.45 kB
          ...                 ...
** Angular Live Development Server is listening on
   http://localhost:4200/ **
```

ANGULAR CLI WORKFLOW

PROJECT STRUCTURE (ng new my-app)



Key Takeaway: Angular CLI accelerates development with automation, consistent workflows, and powerful tooling—helping teams build scalable Angular applications faster.

CREATING AN ANGULAR APPLICATION

ng new asks a few questions, then hands you a working, runnable application.

Run ng new crm-ui

The CLI prompts for routing, stylesheet format, and SSR -- answers become the workspace's starting configuration.

Workspace Is Generated

A full folder tree appears: src/, angular.json, package.json, tsconfig.json, and a working root component.

ng serve to Run It

cd crm-ui && ng serve boots a dev server; the default app renders immediately at localhost:4200.

KEY TAKEAWAY ng new followed by ng serve takes you from an empty folder to a running Angular app in under a minute -- no manual wiring required.

CREATING AN ANGULAR APPLICATION

Angular applications are created using the Angular CLI. The CLI scaffolds the project, installs dependencies, and sets up the initial workspace.

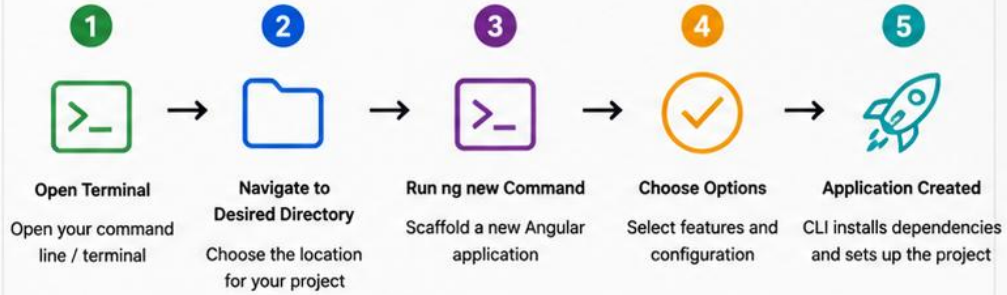
PREREQUISITES

Node.js
Version 18.x or later recommended

Angular CLI
Install globally using npm

npm
Comes with Node.js

CREATE A NEW ANGULAR APPLICATION



COMMON OPTIONS

Option	Description
`--routing`	Adds Angular Router to the application
`--style=scss`	Uses SCSS for styling
`--style=css`	Uses CSS for styling (default)
`--skip-tests`	Skips test files
`--skip-git`	Does not initialize a Git repository
`--ssr`	Enables Server-Side Rendering

INSTALL ANGULAR CLI

```
# Install Angular CLI globally
npm install -g @angular/cli

# Verify installation
ng version
```

EXAMPLE COMMAND

```
ng new employee-portal
? Would you like to add Angular routing? Yes
? Which stylesheet format would you like to use? CSS
? Would you like to enable server-side rendering (SSR)? No
? Would you like to enable strict mode? Yes
✓ Project "employee-portal" created successfully.
```

START THE DEVELOPMENT SERVER

```
cd employee-portal
ng serve
✓ Compiled successfully.
✓ Local:   http://localhost:4200/
```

KEY TAKEAWAY

The Angular CLI simplifies the process of creating and managing Angular applications by automating setup, configuration, and project structure.

TIP

Use the Angular CLI help for more options:
`ng help`

ANGULAR WORKSPACE STRUCTURE

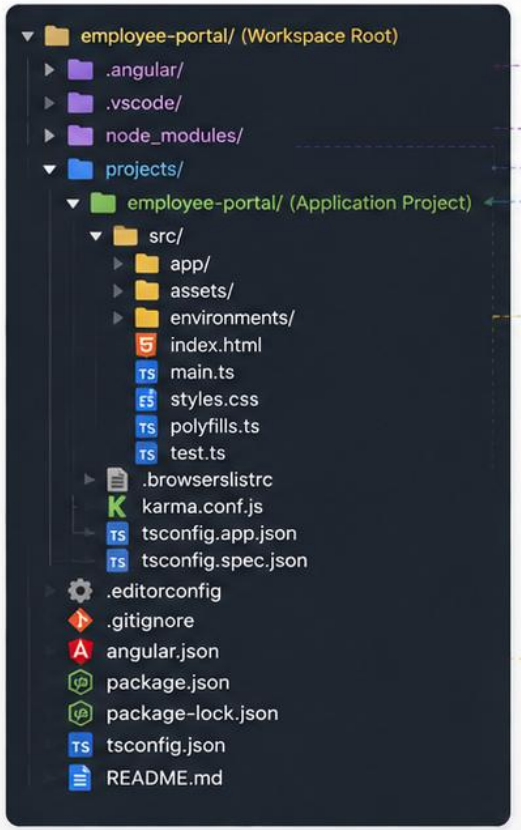
Every generated workspace shares the same top-level files -- learn what each one configures.

File / Folder	Purpose
angular.json	Workspace-wide build, serve, and test configuration -- the CLI reads this to know how to run the app.
package.json	npm dependencies and the CLI's npm scripts (start, build, test).
tsconfig.json	TypeScript compiler options shared by the whole workspace, including strict mode.
src/	All application source code -- components, services, assets, and the entry point.
node_modules/	Installed dependencies -- never edited by hand, never committed to git.

KEY TAKEAWAY angular.json drives how the CLI builds and serves the app; everything you write as a developer lives under src/.

ANGULAR WORKSPACE STRUCTURE

An Angular workspace is a collection of projects and configuration files that manage an Angular application and its build, test, and deployment setup.



.angular/
Cache and build artifacts generated by the Angular CLI.

.vscode/
Editor-specific settings for Visual Studio Code.

node_modules/
Installed npm dependencies.

projects/
Contains one or more Angular projects.

employee-portal/
The main application project.

src/
Source code root of the application.

app/
Components, directives, services, pipes, modules, etc.

assets/
Static assets like images, fonts, and global files.

environments/
Environment-specific configuration files.

index.html
Main HTML file that hosts the Angular application.

main.ts
Application entry point. Bootstraps the root component.

styles.css
Global styles for the application.

Workspace Configuration Files
Define workspace, build, dependencies, TypeScript, and tooling configuration.

KEY WORKSPACE FILES

angular.json	Workspace configuration. Defines projects, build options, architect targets, and CLI settings.
package.json	Project metadata and npm dependencies.
tsconfig.json	Root TypeScript configuration for the workspace.
.gitignore	Specifies intentionally untracked files to ignore by Git.
README.md	Project documentation.

WORKSPACE BENEFITS

Scalability
Manage multiple projects in a single workspace.

Consistency
Centralized configuration ensures uniform builds.

Collaboration
Shared workspace structure improves team productivity.

Maintainability
Clear separation of concerns and organized structure.

KEY TAKEAWAY

Understanding the Angular workspace structure helps you navigate projects, manage configurations, and maintain scalable applications effectively.

TIP
Use Angular CLI commands to generate features and maintain a clean, consistent workspace.

`src` FOLDER ARCHITECTURE

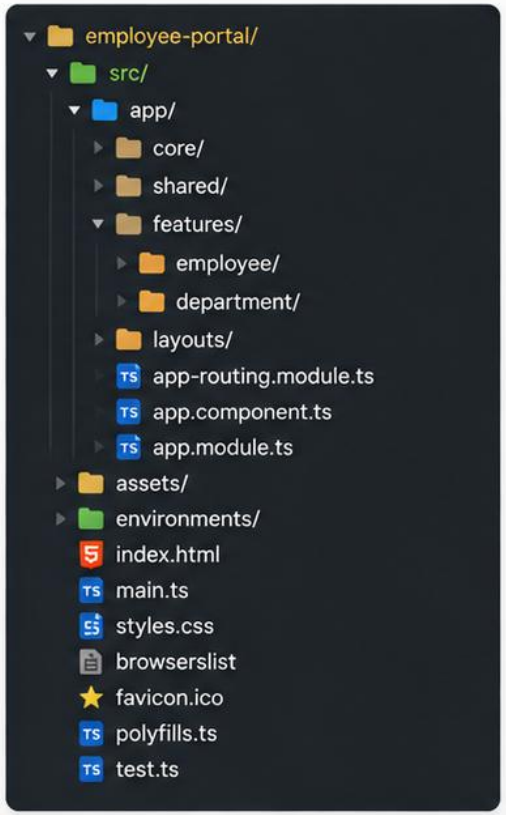
Inside src/, a small, consistent set of files and folders holds the entire application.

Path	Purpose
src/app/	Application components, services, and routes -- almost all of your code lives here.
src/assets/	Static files (images, icons, JSON fixtures) copied as-is into the build output.
src/environments/	Environment-specific config (API base URLs) swapped in at build time.
src/index.html	The single HTML page the whole single-page app boots into.
src/main.ts	The application's entry point -- calls bootstrapApplication with the root component.

KEY TAKEAWAY src/app holds the application itself; src/main.ts and src/index.html are the two files that get it running in a browser.

`src` FOLDER ARCHITECTURE

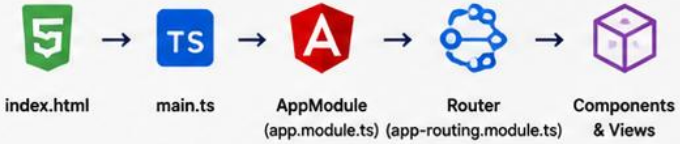
The `src` folder contains all the source code and assets of your Angular application.
It is organized to promote scalability, maintainability, and feature modularity.



WHAT LIVES WHERE?

app/	Root module of the application. Contains components, services, modules, and routing configuration.
core/	Singleton services and core functionality (e.g., auth service, interceptors, guards) used across the entire application.
shared/	Reusable components, directives, pipes, and modules used by multiple features.
features/	Feature modules organized by business domain (e.g., employee, department). Each feature is self-contained.
layouts/	Layout components (e.g., header, sidebar, footer, dashboards) that provide the application shell.
assets/	Static assets such as images, fonts, icons, and i18n files.
environments/	Environment-specific configuration files (e.g., API URLs, production flags).
index.html	Main HTML file that is served by the browser. Hosts the root `<app-root>`.
main.ts	Application entry point. Bootstraps the root Angular module.
styles.css	Global styles applied across the entire application.

TYPICAL FLOW



The browser loads index.html → Angular bootstraps via main.ts → AppModule initializes → Router loads features → Components render views.

BEST PRACTICES

Feature-First Organization
Group code by business features for scalability.

Keep Core Clean
Add only application-wide singleton services in core/.

Reuse from Shared
Place reusable UI and utilities in shared/.

Lazy Load Features
Configure lazy loading for routes to improve performance.

Environment Isolation
Use environments/ for safe configuration management.

KEY TAKEAWAY

A well-structured `src` folder helps build modular, testable, and maintainable Angular applications that grow with your business.

TIP

Use Angular CLI commands (e.g., `ng g feature employee`) to maintain consistent folder architecture.

APPLICATION BOOTSTRAP

One function call in main.ts turns a component class into a running application in the browser.

bootstrapApplication()

Called once in main.ts, it renders the root standalone component into index.html's <app-root>.

App Config

Providers for the router, HTTP client, and other app-wide services are registered here, not in an NgModule.

Rendered in the Browser

Once bootstrap resolves, Angular's change detection takes over and the app becomes interactive.

KEY TAKEAWAY bootstrapApplication(AppComponent, appConfig) in main.ts is the one line that starts every modern standalone Angular app.

TRY IT YOURSELF — EXERCISE 1: PLAN THE ANGULAR WORKSPACE

Reinforces: the Angular CLI, workspace structure, and bootstrap slides you just walked, planned for the lab33-crm-ui application.

STEPS (notes/lab33-workspace-plan.md)

Step	What To Do
1 — Create Command	Write the ng new command for lab33-crm-ui, including routing and style choice.
2 — Folder Map	Name what lives in src/app, and where a customers feature folder would sit.
3 — Bootstrap Path	Trace startup: index.html to main.ts to the root component.
4 — Scope	Plan only — do not run ng new in this pre-lab.

Workspace sketch

```
ng new lab33-crm-ui --routing --style=scss --standalone
src/app/  app.config.ts  app.routes.ts  features/customers/  shared/
index.html <app-root> <- main.ts bootstrapApplication(AppComponent)
```

TRY IT NOW — Complete Exercise 1 before continuing: exercises/exercise-01-workspace-plan.md (workspace: examples/module-33-exercises).

APPLICATION BOOTSTRAP

Bootstrap is the process of starting an Angular application. It initializes the root module (or standalone application), sets up the platform, and renders the root component.

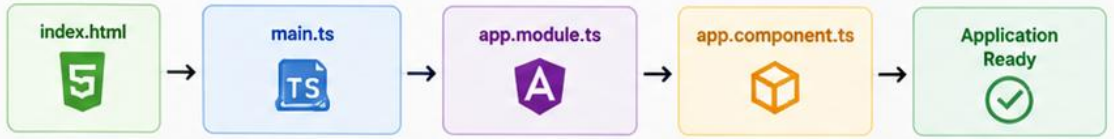
WHAT IS BOOTSTRAP?

Bootstrap is the first step Angular takes to launch your application in the browser.

BOOTSTRAP RESPONSIBILITIES

- Initialize the Angular platform
- Load the root module or standalone configuration
- Create and render the root component
- Enable dependency injection and services
- Set up routing, interceptors, and global configurations

BOOTSTRAP FLOW (NgModule-Based Application)



1. Browser loads index.html
2. Calls platformBrowserDynamic) and bootstraps AppModule
3. AppModule is loaded and configured
4. AppComponent is created and rendered
5. Angular app is running in the browser

main.ts

```ts
import { platformBrowserDynamic } from
  '@angular/platform-browser-dynamic';
import { AppModule } from './app/app.module';

platformBrowserDynamic()
  .bootstrapModule(AppModule)
  .catch(err => console.error(err));
```

app.module.ts

```ts
import { NgModule } from '@angular/core';
import { BrowserModule } from '@angular/platform-browser';
import { AppComponent } from './app.component';

@NgModule({
  declarations: [AppComponent],
  imports: [BrowserModule],
  providers: [],
  bootstrap: [AppComponent]
})
export class AppModule { }
```

TYPICAL BOOTSTRAP FILES

index.html	Entry HTML file that loads the Angular bundle.
main.ts	Application entry point. Bootstraps the root module or standalone app.
app.module.ts	Root NgModule that declares, imports, and bootstraps the root component.
app.component.ts	Root component that serves as the host of the application.
styles.css	Global styles for the application.
environment.ts	Environment-specific configuration.

KEY TAKEAWAY

Bootstrap is the foundation of every Angular application. It connects the browser to your Angular code and brings your UI to life.

TIP

Use browser developer tools to inspect the network requests and confirm that Angular bundle is loaded successfully.

STANDALONE BOOTSTRAP (Anguler 14+)

main.ts

```ts
import { bootstrapApplication } from '@angular/platform-browser';
import { AppComponent } from './app/app.component';

bootstrapApplication(AppComponent);
```

app.config.ts (optional)

```ts
import { ApplicationConfig } from '@angular/core';

export const appConfig: ApplicationConfig = {
  providers: [ /* global providers */ ]
};
```

WHAT IS AN ANGULAR COMPONENT?

The fundamental building block: a TypeScript class paired with an HTML template that renders it.

A Class

Holds state (properties) and behavior (methods) -- plain TypeScript, decorated to become a component.

A Template

The HTML that renders the class's state and reacts to user interaction -- inline or in a separate file.

A Selector

A custom HTML tag (e.g. <app-customer-card>) that lets any parent template place this component.

KEY TAKEAWAY A component is a class plus a template plus a selector -- Angular's compiler wires the three together at build time.

WHAT IS AN ANGULAR COMPONENT?

An Angular component is a reusable building block of an Angular application.
It controls a portion of the user interface (UI) and encapsulates the template, logic, and styles needed to render that UI.

KEY CONCEPTS

UI Building Block
Components are the fundamental building blocks of the UI.

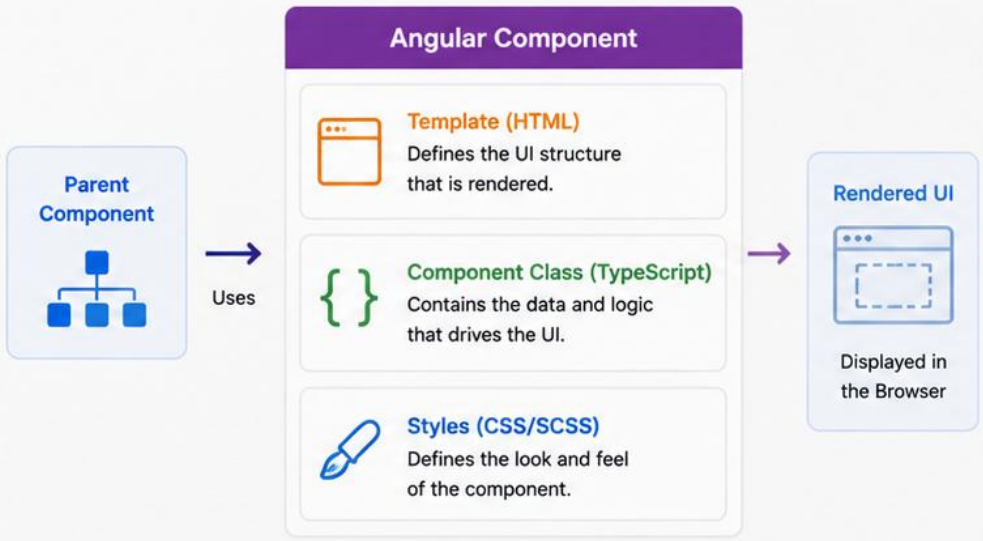
Encapsulation
Each component bundles its template, logic, and styles.

Reusability
Components can be reused across the application.

Composability
Components can be nested inside other components.

Lifecycle Aware
Components have a lifecycle managed by Angular.

COMPONENT OVERVIEW



EXAMPLE COMPONENT USAGE

```
<app-employee-card
  [employee]="emp"
  (select)="onSelect(emp)"
</app-employee-card>
```

- The parent component uses `<app-employee-card>`.
- Data is passed in using `@Input()` binding.
- Events are emitted using `@Output()` binding.

BENEFITS

- Better code organization
- Easier maintenance and testing
- Improved readability
- Supports large, scalable applications
- Encourages separation of concerns

REAL-WORLD ANALOGY

Think of components as Lego bricks. Each brick (component) has its own shape, logic, and style, and can be combined to build complex structures (applications).

KEY TAKEAWAY

Angular components are the heart of an Angular application.
They define what the user sees, how it behaves, and how it looks.

COMPONENT ANATOMY

Four files usually make up one component -- generated together, always in sync.

The Class (.ts)

TypeScript class decorated with @Component -- holds logic, state, and the decorator's metadata.

The Template (.html)

Markup for what the component renders, using Angular's template syntax.

The Styles (.css)

Component-scoped styles that, by default, do not leak out to affect other components.

The Spec (.spec.ts)

A unit test file the CLI scaffolds alongside every generated component.

KEY TAKEAWAY ng generate component customer-card creates all four files together and wires their relationships automatically -- you rarely hand-link them.

Component Anatomy

An Angular component brings together TypeScript logic, HTML template, and styles to render a reusable piece of UI.

Example: employee-card Component

- employee-card/
  - employee-card.component.ts
  - employee-card.component.html
  - employee-card.component.css
  - employee-card.component.scss
  - employee-card.component.spec.ts

Key Idea

A component is a self-contained building block with logic (TypeScript), view (HTML), and styles (CSS/SCSS).

employee-card.component.ts

```ts
import { Component, Input, Output, EventEmitter } from '@angular/core';

@Component({
  selector: 'app-employee-card',
  templateUrl: './employee-card.component.html',
  styleUrls: ['./employee-card.component.scss']
})
export class EmployeeCardComponent {
  @Input() employee!: Employee;
  @Output() view = new EventEmitter<number>();
  onView(id: number) {
    this.view.emit(id);
  }
}
```

Metadata

Inputs

Outputs

Component Logic (Methods)

employee-card.component.html

```html
<div class="card">
  <h3>{{ employee.name }}</h3>
  <p>{{ employee.position }}</p>
  <button (click)="onView(employee.id)">View Details</button>
</div>
```

employee-card.component.scss

```scss
.card {
  border: 1px solid #e0e0e0;
  border-radius: 8px;
  padding: 16px;
  background: #fff;
}
```

The Parts of a Component

Metadata (@Component)
Defines the selector, template, and styles that tell Angular how the component should be used and rendered.

Inputs (@Input)
Receive data from parent components through input properties.

Outputs (@Output)
Emit events to parent components using EventEmitter.

Component Logic
TypeScript class that holds the component state and behavior (methods, properties).

Template (HTML)
Defines the UI structure and binds to the component's data and events.

Styles (CSS / SCSS)
Defines the presentation and layout of the component.

Why it Matters: Understanding component anatomy helps you build reusable, maintainable, and testable UI building blocks.

`@Component` DECORATOR

The decorator is where a plain TypeScript class declares itself as an Angular component.

Property	What It Configures
selector	The custom HTML tag other templates use to place this component, e.g. 'app-customer-card'.
standalone	true means this component declares its own imports -- no enclosing NgModule needed.
imports	Other standalone components, directives, or pipes this component's template depends on.
templateUrl / template	Path to the external HTML file, or an inline template string.
styleUrls / styles	Path(s) to external stylesheet(s), or inline style strings, scoped to this component.

KEY TAKEAWAY @Component's metadata -- selector, standalone, imports, template, styles -- is what turns a plain class into something Angular can render.

@Component DECORATOR

ANGULAR

The **@Component** decorator provides metadata that tells Angular how to create and render a component.

user-card.component.ts

```ts
import { Component } from '@angular/core';

@Component({
  selector: 'app-user-card',
  templateUrl: './user-card.component.html',
  styleUrls: ['./user-card.component.scss'],
  standalone: true,
  imports: [CommonModule],
  changeDetection: ChangeDetectionStrategy.OnPush
})
export class UserCardComponent {
  // component logic
}
```

selector
The CSS selector used to place this component in a template.

templateUrl
Path to the HTML template that defines the component's view.

styleUrls
One or more CSS/SCSS files that style the component.

standalone
Marks the component as standalone (no NgModule required).

imports
Modules or standalone components/directives/pipes used by this component.

changeDetection
Configures Angular's change detection strategy for better performance.

Key Points

- The **@Component** decorator is required for every Angular component.
- It defines the component's view, appearance, and behavior configuration.
- Metadata helps Angular compile and render the component efficiently.

Why it Matters: The @Component decorator is the foundation of every Angular component, connecting the class, template, and styles into a reusable UI building block.

COMPONENT CLASS

Below the decorator, an ordinary TypeScript class holds the component's state and behavior.

Properties

Typed fields holding the component's state -- plain values, or signals for reactive state (Module 34).

Methods

Functions the template calls in response to events, e.g. onSave() bound to a (click) handler.

Constructor & DI

Services are requested as constructor parameters (or via inject()) and Angular supplies them.

KEY TAKEAWAY The class is where the actual logic lives -- the decorator just tells Angular this class is a component; the class itself is ordinary, testable TypeScript.

COMPONENT CLASS

ANGULAR

The component class (TypeScript class) contains the data and logic that drive the component's behavior.

user-card.component.ts

```typescript
import { Component, Input, Output, EventEmitter } from '@angular/core';

@Component({
  selector: 'app-user-card',
  templateUrl: './user-card.component.html',
  styleUrls: ['./user-card.component.scss']
})
export class UserCardComponent {
  @Input() user!: User;
  @Output() edit = new EventEmitter<User>();
  isLoading = false;
  errorMessage: string | null = null;

  ngOnInit(): void {
    this.isLoading = true;
  }

  onEdit(): void {
    this.edit.emit(this.user);
  }
}
```

Class Declaration
Defines the component class that Angular will instantiate and manage.

Inputs
Receive data from parent components using `@Input` properties.

Outputs
Emit events to parent components using `@Output` and `EventEmitter`.

Component State
Holds local data and UI state used by the template.

Lifecycle Hook
Methods like `ngOnInit()` run at specific points in the component lifecycle.

Component Methods
Custom methods handle user actions and business logic.

Key Points

- The component class is written in TypeScript.
- It contains properties for data, state, and behavior.
- It interacts with the template and other components.
- Angular creates one instance of the class per component in the DOM.

Why it Matters: A well-structured component class keeps your UI logic organized, reusable, and testable.

ANGULAR TEMPLATES

HTML, extended with Angular-specific syntax, that renders a component's state to the screen.

Still HTML

Any valid HTML works in a template -- Angular syntax is additive, not a replacement language.

Data-Aware

Templates read component properties directly by name -- no separate templating language to learn.

Compiled, Not Interpreted

Angular's compiler turns templates into efficient JavaScript at build time, not parsed at runtime.

KEY TAKEAWAY A template is HTML plus a small, purpose-built syntax for binding data and handling events -- covered in full over the next several slides.

ANGULAR TEMPLATES

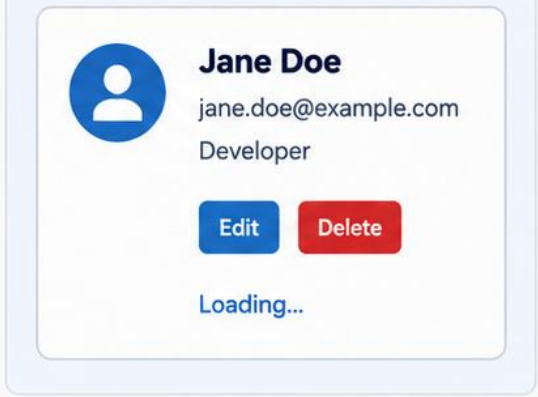
ANGULAR

Angular templates define the HTML structure of the UI and bind it to the component class data and behavior.

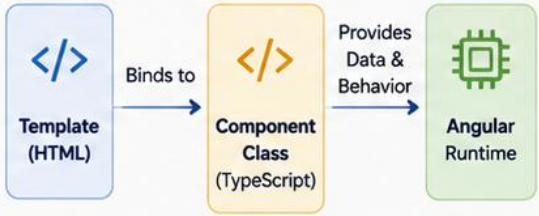
user-card.component.html

```
1  <div class="card">
2    <h3>{{ user.name }}</h3>
3    <p class="email">{{ user.email }}</p>
4    <p class="role">{{ user.role }}</p>
5
6    <button (click)="edit.emit(user)">Edit</button>
7    <button (click)="delete.emit(user.id)" class="danger">
8      Delete
9    </button>
10   <div *ngIf="isLoading">Loading...</div>
11   <div *ngIf="errorMessage" class="error">
12     {{ errorMessage }}
12   </div>
13 </div>
```

Rendered Output in Browser



Template Binding Flow



Key Points

- Templates define what the user sees.
- They bind to the component class to display data and handle user interactions.
- Angular's template syntax makes the UI dynamic and responsive.
- Templates are compiled by Angular and rendered in the browser.

Template Features

Interpolation
Display component data.
`{{ user.name }}`

Property Binding
Bind element properties to data.
`[value]="user.name"`

Event Binding
Listen to userer events.
`(click)="edit.emit(user)"`

Two-Way Binding
Sync data between view and component.
`[(ngModel)]="user.name"`

Structural Directives
Add or remove elements from the DOM.
`*ngIf="isLoading"`

Attribute Directives (e.g., **ngClass**)
Change the appearance/behavior of elements.
`[ngClass]="{ active: isActive }"`

Why it Matters: Templates connect the UI structure with the component logic, enabling dynamic, interactive, and maintainable user interfaces.

TEMPLATE SYNTAX

A small set of symbols -- {{ }}, [ ], ( ), and control-flow blocks -- covers nearly every template need.

Syntax	Purpose
{{ expression }}	Interpolation -- renders a value as text inside an element.
[property]="expr"	Property binding -- sets a DOM property or component @Input from an expression.
(event)="handler()"	Event binding -- calls a method when the named DOM or component event fires.
[(ngModel)]="value"	Two-way binding -- combines property and event binding into one syntax.
@if / @for / @switch	Modern control-flow blocks (Angular 17+) for conditional and repeated rendering.

KEY TAKEAWAY Four binding symbols plus the new @if/@for control-flow blocks cover essentially every Angular template you will write.

TEMPLATE SYNTAX

ANGULAR

Angular templates use special syntax to bind data, handle events, and control the DOM.

Syntax	Description	Example
{{}} Interpolation	Displays the value of a component property.	`{{ userName }}`
[] Property Binding	Binds a DOM element property to a component property.	`[disabled]="isDisabled"`
() Event Binding	Listens to a DOM event and executes a component method.	`(click)="onSave()"`
[()] Two-Way Binding	Binds data between the component and the view (banana-in-a-box).	`[(ngModel)]="userName"`
* Structural Directives	Add or remove elements from the DOM based on a condition.	`*ngIf="isLoggedIn"` `*ngFor="let item of items"`
ng Attribute Directives	Change the appearance or behavior of an element.	`[ngClass]="{ active: isActive }"` `[ngStyle]="{ color: textColor }"`

Example: user-card.component.html

```
1  <div class="card" *ngIf="user">
2     <h3>{{ user.name }}</h3>
3     <p>Email: {{ user.email }}</p>
4     <input [value]="user.email" (input)="onEmailChange($event)" />
5     <button (click)="onSave()" [disabled]="!isFormValid">Save</button>
6     <ul>
7        <li *ngFor="let role of user.roles"> {{ role }}</li>
8     </ul>
9     <p [ngClass]="{ active: user.active }">Status: {{ user.active }}</p>
10 </div>
```

Rendered Output in Browser

Jane Doe
Email: jane.doe@example.com

jane.doe@example.com

Save

Roles
- Developer
- Analyst
- Team Lead

Status: true

Key Points
- Template syntax connects the UI with the component class.
- It makes the UI dynamic and interactive.
- Angular compiles the template into optimized JavaScript.

Syntax Summary

{{ }}	[]	()	[()]	*	ng
Interpolation	Property Binding	Event Binding	Two-Way Binding	Structural Directives	Attribute Directives

Why it Matters: Understanding template syntax is essential to build dynamic, responsive, and maintainable Angular applications.

DATA BINDING IN ANGULAR TEMPLATES

Four binding forms, one underlying idea: keep the template and the component class in sync.

Interpolation

{{ customer.name }} renders a component value as text -- one-way, class to view.

Property Binding

[disabled]="isSaving" sets a DOM property or child @Input from a class value -- one-way, class to view.

Event Binding

(click)="onSave()" calls a class method when an event fires -- one-way, view to class.

Two-Way Binding

[(ngModel)]="name" combines property and event binding -- value flows both directions at once.

KEY TAKEAWAY Interpolation and property binding push data outward; event binding pulls it back in; two-way binding does both in a single expression.

INTERPOLATION

ANGULAR

Interpolation displays the value of a component property or expression inside the template using double curly braces {{ }}.

user-card.component.ts

```ts
import { Component } from '@angular/core';

@Component({
  selector: 'app-user-card',
  templateUrl: './user-card.component.html',
  styleUrls: ['./user-card.component.scss']
})
export class UserCardComponent {
  user = {
    name:  'Jane Doe',
    email: 'jane.doe@example.com',
    role:  'Developer'
  };
}
```

user-card.component.html

```html
<div class="card">
  <h3>{{ user.name }}</h3>
  <p class="email">{ user.email }}</p>
  <p class="role">{{ user.role }}</p>

  <p class="title">
    Hello, {{ user.name }}!
  </p>
</div>
```

Rendered Output in Browser

Jane Doe
jane.doe@example.com
Developer

Hello, Jane Doe!

How Interpolation Works

- **Component Class** (TS) — Holds data (e.g., user.name)
- **Template** (HTML) — Uses {{ }} to bind data
- **Angular Runtime** — Replaces {{ }} with actual values
- **Browser** — Displays the rendered output

Key Points

- Use double curly braces {{ }} to display values.
- Works with component properties and expressions.
- Updates automatically when the underlying data changes.
- Safe from XSS by default (values are HTML-escaped).

Common Examples

`{{ user.name }}`	`{{ 2 + 2 }}`	`{{ user.email.toLowerCase() }}`	`{{ user.roles.join(', ') }}`
Display property value	Evaluate expression	Call method	Complex expression

Why it Matters: Interpolation connects your data to the UI, making your application dynamic and responsive to changes.

PROPERTY BINDING

ANGULAR

Property binding sets an element property to a value from the component class using square brackets [ ].

user-card.component.ts

```ts
import { Component } from '@angular/core';

@Component({
  selector: 'app-user-card',
  templateUrl: './user-card.component.html',
  styleUrls: ['./user-card.component.scss']
})
export class UserCardComponent {
  user = {
    name: 'Jane Doe',
    email: 'jane.doe@example.com',
    profileUrl: 'https://example.com/profile/jane.jpg',
    isActive: true
  };
}
```

user-card.component.html

```html
<div class="card">
  <img class="avatar"
      [src]="user.profileUrl"
      [alt]="user.name"
      width="80" />

  <h3 class="name" [textContent]="user.name"></h3>
  <p class="email">Email: <span
   [textContent]="user.email"></span></p>
  <span class="status" [class.active]="user.isActive">
    {{ user.isActive ? 'Active' : 'Inactive' }}
  </span>
</div>
```

Rendered Output in Browser

Jane Doe

Email: jane.doe@example.com

Active

Key Points

- Use square brackets [ ] to bind to DOM element properties.
- Angular updates the property value when the component data changes.
- Property binding is one-way: component → view.
- Prevents XSS by default (values are treated as data, not HTML).

How Property Binding Works

Component Class	Template	Angular Runtime	Browser
Holds the data (e.g., user.name)	Uses [property] to bind data	Sets the element property value	Updates the DOM with the value

Common Property Binding Examples

Set Image Source	Set Input Value	Disable Button	Toggle CSS Class
`<img [src]="imageUrl">`	`<input [value]="name">`	`<button [disabled]="isDisabled">`	`<div [class.active]="isActive">`
Binds the src property	Binds the value property	Binds the disabled property	Adds class when condition is true

Why it Matters: Property binding keeps your UI in sync with application state, ensuring a dynamic, interactive, and responsive user experience.

EVENT BINDING

ANGULAR

Event binding listens to DOM events (like click, input, submit) and executes a method in the component using parentheses ( ).

user-card.component.ts

```ts
import { Component } from '@angular/core';

@Component({
  selector: 'app-user-card',
  templateUrl: './user-card.component.html',
  styleUrls: ['./user-card.component.scss']
})
export class UserCardComponent {
  user = {
    name: 'Jane Doe',
    email: 'jane.doe@example.com'
  };

  isActive = false;

  toggleActive(): void {
    this.isActive = !this.isActive;
  }
}
```

user-card.component.html

```html
<div class="card">
  <h3>{{ user.name }}</h3>
  <p>{{ user.email }}</p>

  <button class="btn" (click)="toggleActive()">
    {{ isActive ? 'Deactivate' : 'Activate' }}
  </button>

  <p class="status" [class.active]="isActive">
   Status: {{ isActive ? 'Active' : 'Inactive' }}
  </p>
</div>
```

Rendered Output in Browser

Jane Doe

jane.doe@example.com

Deactivate

Status: Active

How Event Binding Works

User Action — User clicks the button

→

Event Binding — (click) listens for the event

→

Component Method — toggleActive() is executed

→

UI Update — Component state and UI update

Key Points

- Use parentheses ( ) to bind DOM events to component methods.
- Common events: click, input, change, submit, keyup, mouseover, etc.
- Angular runs the bound method inside the component class.
- Keeps UI interactive and connected to application logic.

Common Event Binding Examples

Click Event

```
<button (click)="save()">
Save</button>
```

Input Event

```
<input (input)="onInput($event)">
```

Change Event

```
<select (change)="onChange($event)">
</select>
```

Submit Event

```
<form (submit)="onSubmit($event)">
</form>
```

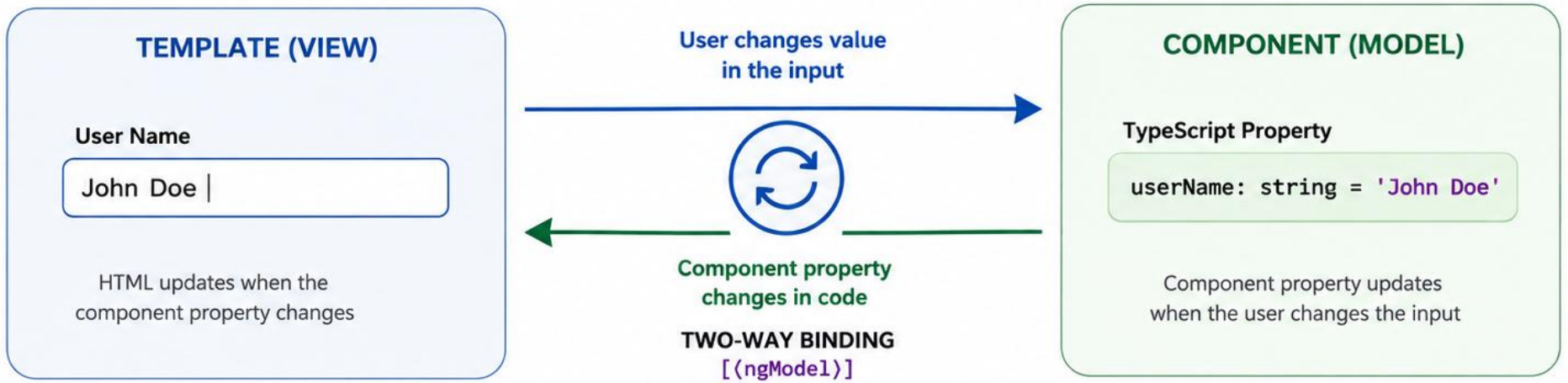
Why it Matters: Event binding connects user actions to your component logic, creating dynamic, responsive, and interactive applications.

ANGULAR COMPONENT ARCHITECTURE

Two-Way Binding

ANGULAR

Two-way binding synchronizes data between the component (TypeScript) and the template (HTML).
When the user changes the value in the UI, the component property updates. When the property changes in code, the UI updates.



KEY POINTS

- Two-way binding uses the syntax: `[(ngModel)]="property"`
- Changes in the input update the component property.
- Changes in the component property update the input value.
- Built using a combination of property binding [ ] and event binding ( ).
- Ideal for forms and user input scenarios.

EXAMPLE

template.component.html

```
<div class="form-group">
  <label for="username">User Name</label>
  <input
    id="username"
    type="text"
    [(ngModel)]="userName"
    class="form-control"
    placeholder="Enter user name"
  />
</div>
```

template.component.ts

```
import { Component } from '@angular/core';
import { FormsModule } from '@angular/forms';

@Component({
  selector: 'app-template',
  standalone: true,
  imports: [FormsModule],
  templateUrl: './template.component.html'
})
export class TemplateComponent {
  userName: string = 'John Doe';
}
```

TRY IT YOURSELF — EXERCISE 2: MAP COMPONENT ANATOMY AND BINDING

Reinforces: component anatomy, the @Component decorator, and the four binding forms you just covered, applied to a customer list.

STEPS (notes/lab33-component-anatomy.md)

Step	What To Do
1 — Decorator Fields	List what @Component declares: selector, template, styles, imports.
2 — Binding Table	Give one example each of interpolation, property, event, and two-way binding.
3 — Render Amina	Write the template line that renders CUS-1001's name from a class field.
4 — Direction	For each binding form, say whether data flows to the view, from it, or both.

The four bindings

```
{{ customer.name }}          interpolation   class -> view
[disabled]="isLoading"       property        class -> view
(click)="select(c)"          event           view  -> class
```

TRY IT NOW — Complete Exercise 2 before continuing: exercises/exercise-02-component-anatomy.md (workspace: examples/module-33-exercises).

COMPONENT INPUTS

A parent passes data down into a child component through explicitly declared input properties.

@Input() Decorator

The classic syntax: @Input() customer!: Customer; declares a settable property on the class.

input() Signal Function

The modern signal-based syntax: customer = input<Customer>(); returns a readonly signal.

Required Inputs

input.required<Customer>() (or @Input({ required: true })) fails to compile if the parent forgets to pass it.

KEY TAKEAWAY Inputs are how a child component receives data from its parent -- prefer the newer input() signal function in new Angular code.

ANGULAR COMPONENT ARCHITECTURE

Component Inputs

Component inputs allow a parent component to pass data to a child component.
The child component receives the data through a property decorated with `@Input()`.

PARENT COMPONENT
app-parent

Responsibilities:

- Holds the data
- Passes data to child
- Controls the child's input

```
<app-child [title]="pageTitle">
</app-child>
```

Data flows from Parent to Child

Binding: `[title]`

Value: `parent.pageTitle`

CHILD COMPONENT
app-child

Responsibilities:

- Receives data from parent
- Displays or uses the data

Input Property:

```
@Input() title: string;
```

KEY POINTS

- Use `@Input()` to receive data in a child component.
- Parent binds data using property binding syntax: `[property]`.
- Inputs are read-only in the child component.
- Use meaningful names for `@Input()` properties.
- Supports one-way data flow (Parent → Child).

PARENT TEMPLATE (app-parent.component.html)

```
<h2>Parent Component</h2>

<p>Page Title: {{ pageTitle }}</p>

<app-child [title]="pageTitle"></app-child>
```

PARENT COMPONENT (app-parent.component.ts)

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-parent',
  templateUrl: './app-parent.component.html'
})

export class AppParentComponent {
  pageTitle: string = 'Employee Dashboard';

}
```

CHILD COMPONENT (app-child.component.ts)

```
import { Component, Input } from '@angular/core';

@Component({
  selector: 'app-child',
  template: `
    <div class="child-box">
      <h3>{{ title }}</h3>
    </div>`
})
export class AppChildComponent {
  @Input() title: string = '';
}
```

COMPONENT OUTPUTS

A child notifies its parent of something that happened by emitting a custom event.

@Output() + EventEmitter

The classic syntax: @Output() selected = new EventEmitter<Customer>();

output() Function

The modern equivalent: selected = output<Customer>(); -- simpler, no EventEmitter import needed.

Emitting an Event

this.selected.emit(this.customer) fires the event; the parent listens with (selected)="...".

KEY TAKEAWAY Outputs are the reverse of inputs: a child announces an event upward, and the parent decides what to do about it -- the child never reaches up directly.

ANGULAR COMPONENT ARCHITECTURE

Component Outputs

Component outputs allow a child component to send data or events to its parent component.
The child exposes an event using `@Output()` and `EventEmitter<T>()`.

CHILD COMPONENT
app-child

Responsibilities:

- Performs an action (e.g., button click)
- Emits an event with data
- Notifies parent component

Output Event:

```
@Output() itemSelected = new
          EventEmitter<string>();
```

Event flows from Child to Parent

Binding: `(itemSelected)`

Event Data: `selectedItem`

PARENT COMPONENT
app-parent

Responsibilities:

- Listens to the child's event
- Receives the data emitted by the child
- Updates its own state or UI

Event Handler:

```
onItemSelected(item: string): void {
  this.selectedItem = item;
}
```

KEY POINTS

- Use `@Output()` with `EventEmitter` to expose events.
- Child emits events to notify the parent.
- Parent listens using event binding syntax: `(eventName)`.
- Enables Child → Parent communication.
- Supports sending data along with the event.

CHILD COMPONENT (app-child.component.ts)

```
import { Component, Output, EventEmitter }
 from '@angular/core';

@Component({
  selector: 'app-child',
  templateUrl: './app-child.component.html'
})
export class AppChildComponent {
  @Output() itemSelected =
      new EventEmitter<string>();

  onSelect(item: string): void {
    this.itemSelected.emit(item);
  }
}
```

CHILD TEMPLATE (app-child.component.html)

```
<div class="child-box">
  <h3>Child Component</h3>
  <button (click)="onSelect('Laptop')">
    Select Laptop
  </button>
  <button (click)="onSelect('Phone')">
    Select Phone
  </button>
</div>
```

PARENT TEMPLATE (app-parent.component.html)

```
<div class="parent-box">
  <h2>Parent Component</h2>
  <app-child (itemSelected)="onItemSelected($event)
                    "></app-child>

  <p>Selected Item: {{ selectedItem }}</p>
</div>
```

PARENT-CHILD COMMUNICATION

Inputs down, outputs up -- the one rule that keeps a component tree predictable at any size.

Data Flows Down

Parents pass data into children via inputs -- children never reach up to read parent state directly.

Events Flow Up

Children notify parents of what happened via outputs -- parents decide what response, if any, follows.

Encapsulation Preserved

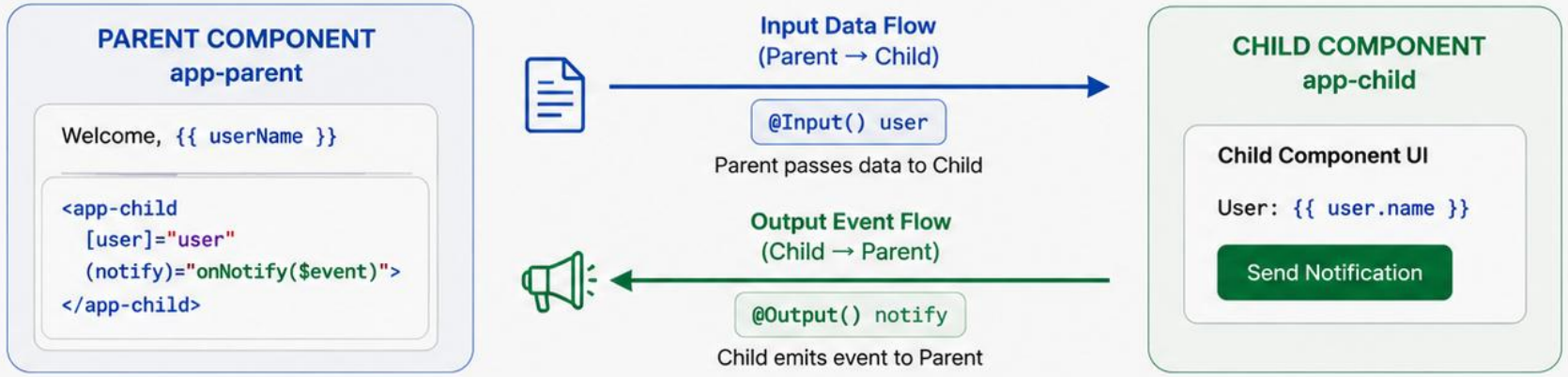
Neither side needs to know the other's internals -- only the input/output contract between them.

KEY TAKEAWAY "Inputs down, outputs up" is not a style preference -- it is the rule that makes a component reusable in a context its author never anticipated.

ANGULAR COMPONENT ARCHITECTURE

Parent–Child Communication

Components communicate using **Inputs** (Parent → Child) and **Outputs** (Child → Parent).
This creates a **unidirectional data flow** that keeps the application predictable and easier to maintain.



KEY POINTS

- Use `@Input()` to pass data from Parent to Child.
- Use `@Output()` with `EventEmitter` to send data from Child to Parent.
- Data flows in one direction at a time.
- Keeps components reusable and loosely coupled.
- Ideal for building maintainable and scalable applications.

PARENT TEMPLATE (app-parent.component.html)

```
<h2>Welcome, {{ userName }}</h2>

<app-child
  [user]="user"
  (notify)="onNotify($event)">
</app-child>

<p>Last notification: {{ message }}</p>
```

PARENT COMPONENT (app-parent.component.ts)

```
export class AppParentComponent {
  user = { name: 'John Doe' };
  message = '';

  onNotify(event: string): void {
    this.message = 'Child says: ' + event;
  }
}
```

CHILD COMPONENT (app-child.component.ts)

```
import { Component, Input, Output,
  EventEmitter } from '@angular/core';

@Component({
  selector: 'app-chiid',
  templateUrl: './app-child.component.html'
})
export class AppChildComponent {
  @Input() user!: { name: string };
  @Output() notify = new EventEmitter<string>();

  sendNotification() {
    this.notify.emit('Hello from child!');
  }
}
```

DATA FLOW SUMMARY

- Parent → Child : `@Input()`
- Child → Parent : `@Output()`

BEST PRACTICE

Keep child components dumb (presentational) and let parent components handle data and business logic.

COMPONENT COMPOSITION

Complex screens are built by nesting simple components, not by making one component do everything.

Small, Focused Pieces

CustomerCard, StatusBadge, and SearchBar each do one thing and compose into CustomerListPage.

Content Projection

<ng-content> lets a wrapper component (e.g. Card) render whatever markup its caller places inside it.

Trees, Not Monoliths

A 'page' component is rarely more than a layout that arranges smaller components -- it renders little content itself.

KEY TAKEAWAY Build screens by composing small components, not by growing one component's template until it does everything.

ANGULAR COMPONENT ARCHITECTURE

Component Composition

ANGULAR

Component composition is the practice of building complex UIs by combining small, reusable components together.
A parent component includes child components in its template to form a UI hierarchy.

KEY BENEFITS

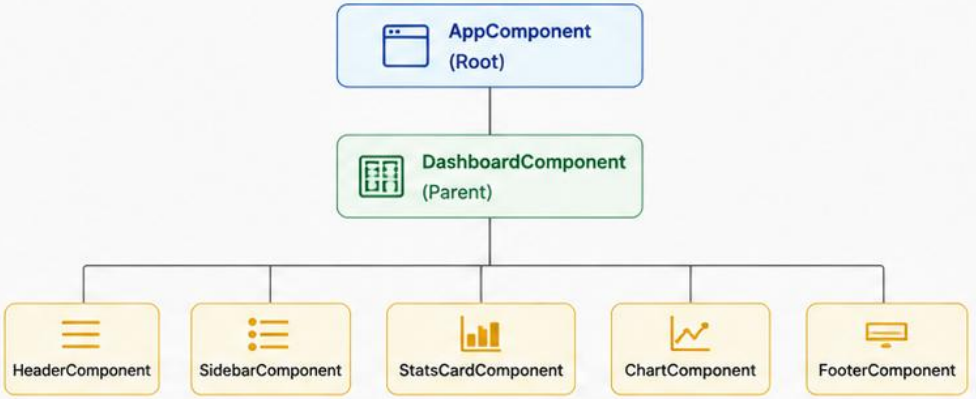
- Promotes reusability and modularity
- Simplifies maintenance and testing
- Encourages separation of concerns
- Builds scalable and readable UIs
- Supports team collaboration

COMMON EXAMPLE

A dashboard page composed of:

- HeaderComponent
- SidebarComponent
- StatsCardComponent
- ChartComponent
- FooterComponent

COMPONENT COMPOSITION HIERARCHY



HOW IT WORKS

1. AppComponent renders DashboardComponent.
2. DashboardComponent composes smaller child components in its template.
3. Data flows down using @Input().
4. Events or data flow up using @Output().

PARENT TEMPLATE (dashboard.component.html)

```
<app-header title="Analytics Dashboard"></app-header>

<div class="layout">
  <app-sidebar></app-sidebar>
  <main class="content">
    <app-stats-card></app-stats-card>
    <app-chart [data]="chartData"></app-chart>
  <main>
</div>
<app-footer></app-footer>
```

CHILD COMPONENT USAGE

```
<!-- Passing data to child -->
<app-stats-card [stats]="statsData"></app-stats-card>

<app-chart [data]="chartData" (selected)="onSelect($event)">
</app-chart>

<!-- Receiving event from child -->
<app-chart (selected)="handleSelection($event)"></app-chart>
```

BEST PRACTICES

- Keep components small and focused
- Use @Input() for data into children
- Use @Output() for events to parent
- Build reusable UI building blocks
- Follow a consistent folder structure

TRY IT YOURSELF — EXERCISE 3: DESIGN THE INPUT/OUTPUT CONTRACT

Reinforces: component inputs, outputs, parent-child communication, and composition as you just saw them, designed for the CRM list and row.

STEPS (notes/lab33-io-contract.md)

Step	What To Do
1 — Child Inputs	Name what CustomerRowComponent receives, with its TypeScript type.
2 — Child Outputs	Name the event it emits upward and what the payload carries.
3 — Parent Template	Write the parent's tag wiring both the input and the output.
4 — No Reach-Around	State the rule: a child never mutates its parent's data directly.

Parent-child wiring

```
child : @Input() customer!: Customer   @Output() selected = new EventEmitter<Customer>()
parent: <app-customer-row [customer]="c" (selected)="onSelect($event)" />
data flows down via inputs; changes travel up via events -- never sideways
```

TRY IT NOW — Complete Exercise 3 before continuing: exercises/exercise-03-input-output-contract.md (workspace: examples/module-33-exercises).

STANDALONE COMPONENTS

Since Angular 17, standalone is the default -- components declare their own dependencies directly.

No NgModule Required

A standalone component lists its template dependencies in its own 'imports' array.

Import What You Use

imports: [CommonModule, CurrencyPipe, CustomerCardComponent] -- explicit, not inherited from a module.

The Modern Default

New CLI-generated components (Angular 17+) are standalone automatically -- no flag needed.

KEY TAKEAWAY Standalone components are self-describing: everything a template needs is listed in that component's own imports array, not inherited from an NgModule.

ANGULAR COMPONENT ARCHITECTURE

Standalone Components

Standalone components are self-contained and do not require an NgModule.
They simplify the architecture and improve maintainability by using direct imports.

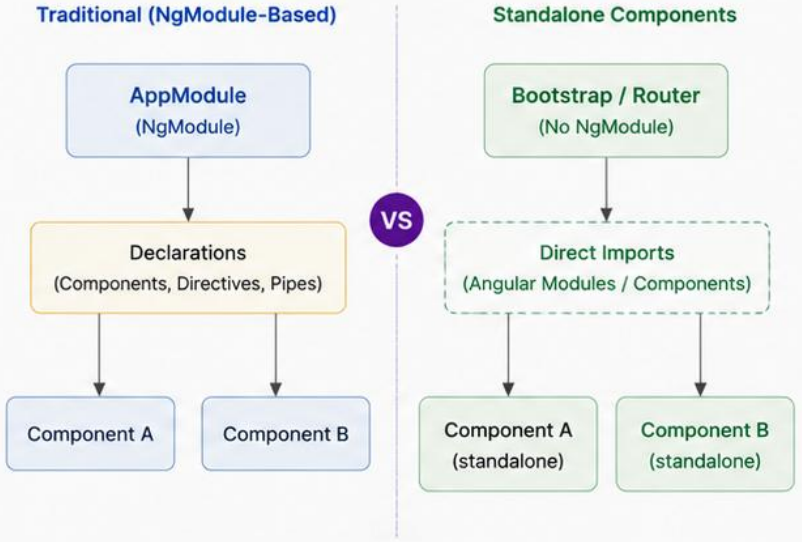
KEY BENEFITS

- No need for NgModule declarations.
- Simpler and more lightweight.
- Faster development with direct imports.
- Improved tree-shaking and performance.
- Easier to understand and test.

HOW IT WORKS

1. Mark the component as standalone: true.
2. Import required Angular modules and other standalone components in the imports array.
3. Use the component directly in templates or routing.
4. No declarations in any NgModule.

ARCHITECTURE COMPARISON



STANDALONE COMPONENT EXAMPLE

```
import { Component } from '@angular/core';
import { CommonModule } from '@angular/common';
import { RouterLink } from '@angular/router';

@Component({
  selector: 'app-hello',
  standalone: true,
  imports: [CommonModule, RouterLink],
  template: `
    <div class="hello">
      <h2>Hello, Standalone!</h2>
       <a routerLink="/home">So Home</a>
   </div>`,
  styles: ['.hello { color: #1976d2; }"]
})
export class HelloComponent {}
```

USING A STANDALONE COMPONENT

```
<!-- In a parent template -->
<app-hello></app-hello>
```

BEST PRACTICES

- Prefer standalone components for new applications.
- Group related imports in shared constants to keep code clean.
- Keep components small, focused and reusable.
- Use lazy loading with standalone components for better performance.
- Follow a consistent folder structure.

ANGULAR MODULES – CONCEPTUAL OVERVIEW

NgModules were Angular's original organizing unit -- still worth understanding to read existing codebases.

declarations

Lists every component, directive, and pipe that belongs to this module.

imports

Other NgModules whose exported components/directives this module's templates need.

bootstrap

Only the root module specifies which component starts the app (AppComponent).

KEY TAKEAWAY An NgModule groups related components and declares what they need -- the same organizing job standalone imports now do per-component, at coarser granularity.

ANGULAR ARCHITECTURE

Angular Modules – Conceptual Overview

Angular modules **(NgModules)** organize the application into cohesive blocks of functionailty.
They group related components, directives, pipes, and services and manage dependencies.

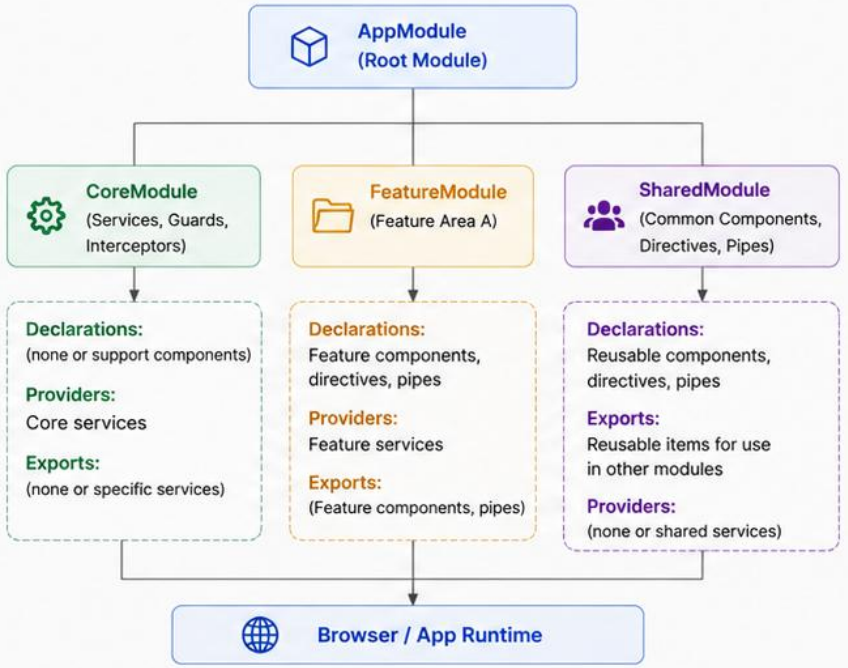
WHY USE MODULES?

- Organize code into logical feature areas
- Encapsulate and scope functionality
- Manage dependencies and visibility
- Promote scalability and maintainability
- Enable lazy loading for performance

MODULE METADATA

- **declarations** – Components, directives, and pipes owned by the module
- **imports** – Other modules whose exported members are needed
- **exports** – Declarations made available to other modules
- **providers** – Services available within the module (injectors)
- **bootstrap** – Root component(s) to launch the application

MODULE-BASED APPLICATION STRUCTURE



EXAMPLE: APPMODULE

```
import { NgModule } from '@angular/core';
import { BrowserModule } from '@angular/platform-browser';
import { AppComponent } from './app.component';
import { SharedModule } from './shared/shared.module';
import { CoreModule } from './core/core.module';
import { FeatureModule } from './features/feature.module';

@NgModule({
  declarations: [AppComponent],
  imports: [BrowserModule, CoreModule, SharedModule, FeatureModule],
  providers: [],
  bootstrap: [AppComponent]
})
export class AppModule { }
```

TYPES OF MODULES

- **Root Module** (e.g., AppModule)
  Bootstraps the application and loads core modules.
- **Core Module**
  Contains singleton services and application-wide logic.
- **Feature Module**
  Encapsulates a specific feature or business capability.
- **Shared Module**
  Holds reusable components, directives, and pipes.

BEST PRACTICES

- Keep modules focused and cohesive.
- Use a SharedModule for reusable UI elements.
- Use lazy-loaded feature modules for scalability.
- Avoid declaring the same item in multiple modules.
- Prefer standalone components for new applications when modules are not needed.

STANDALONE VS NGMODULE-BASED ARCHITECTURE

Same underlying framework, two different ways of declaring what a component depends on.

Aspect	Standalone (Modern Default)	NgModule-Based (Legacy)
Declaring a component	standalone: true (implied), no owning module	Must be declared in exactly one NgModule
Template dependencies	Listed in the component's own 'imports'	Inherited from the owning module's 'imports'
Bootstrapping	bootstrapApplication(AppComponent, config)	platformBrowserDynamic().bootstrapModule(AppModule)
New CLI projects	Default since Angular 17	Opt-in via --no-standalone flag
This course's approach	Used throughout crm-ui	Covered for reading legacy code only

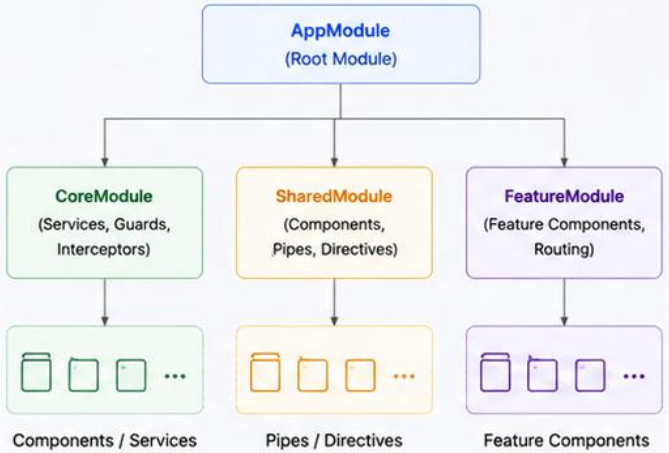
KEY TAKEAWAY Standalone components are self-contained and are what crm-ui will use throughout; NgModules still power a large share of existing production Angular code.

ANGULAR COMPONENT ARCHITECTURE

Standalone vs NgModule-Based Architecture

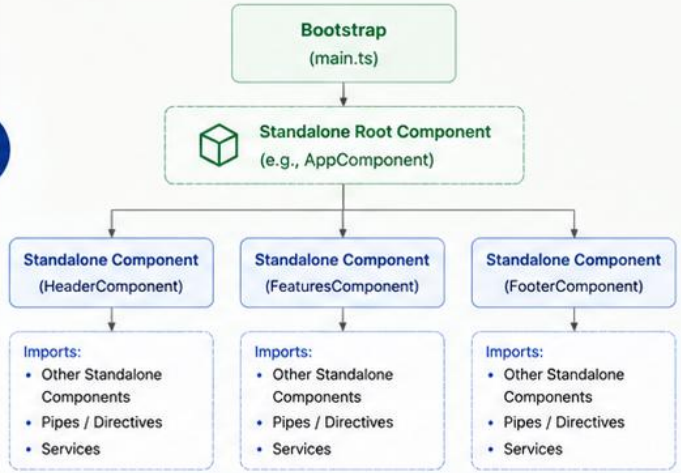
Angular supports two ways to build applications: the traditional NgModule-based approach and the modern Standalone component approach.
Both are valid—choose based on project size, team preference, and scalability needs.

NGMODULE-BASED ARCHITECTURE



VS

STANDALONE ARCHITECTURE



No NgModules required. Components import what they need.

KEY DIFFERENCES

Aspect	NgModule-Based	Standalone
Structure	Organized by NgModules	Components are self-contained
Declarations	Declared in NgModules	No declarations needed
Imports	Import modules	Import what you use directly in component
Boilerplate	More setup	Less setup
Lazy Loading	Module-based	Routle-based with standalone routes
Best For	Large, legacy apps	New projects, simpler architecture

WHEN TO USE WHAT

NgModule-Based
- Large enterprise applications
- Existing legacy codebases
- Teams comfortable with modules

Standalone Architecture
- New applications
- Smaller to medium projects
- Teams prefering modern, lightweight patterns

BENEFITS SUMMARY

NgModule-Based
- Clear separation of concerns
- Module-level encapsulation
- Mature ecosystem and documentation

Standalone Architecture
- Simpler and less boilerplate
- Better tree-shaking and performance
- More explicit dependencies
- Faster development experience

MIGRATION PATH (IF NEEDED)

Best Practice: Use Standalone for new features and components. Mix both approaches during transition if needed.

COMPONENT LIFECYCLE OVERVIEW

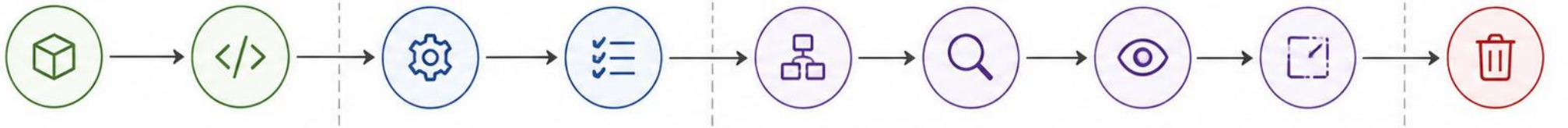
Angular calls specific methods on a component at specific moments -- from creation to destruction.

Hook	Called When
ngOnChanges	Before ngOnInit, and whenever a bound @Input value changes.
ngOnInit	Once, after the first ngOnChanges -- the standard place to fetch initial data.
ngAfterViewInit	Once, after the component's own view (and child views) have been fully initialized.
ngOnDestroy	Once, right before Angular removes the component -- the place to clean up subscriptions.

KEY TAKEAWAY ngOnInit and ngOnDestroy are the two hooks used almost constantly; the others solve narrower, specific problems.

Component Lifecycle Overview

Angular components go through a series of lifecycle phases from creation to destruction.



CREATION

constructor()
Class is instantiated and dependencies are injected.

ngOnChanges()
Called when input properties change.

INITIALIZATION

ngOnInit()
Called once after the first ngOnChanges(). Ideal place for initialization logic.

ngDoCheck()
Custom change detection logic. Called frequently.

CONTENT & VIEW CHECK

ngAfterContentInit()
Called after content (ng-content) has been projected the first time.

ngAfterContentChecked()
Called after every check of projected content.

ngAfterViewInit()
Called after the component's view and child views are initialized.

ngAfterViewChecked()
Called after every check of the component's view and child views.

DESTRUCTION

ngOnDestroy()
Cleanup logic. Unsubscribe from observables and release resources.

LIFECYCLE FLOW

- Angular calls these hooks at specific points during a component's life.
- Not all hooks are needed in every component.
- Implement only what your component requires.

Tip: Use lifecycle hooks to manage data, resources, and interactions at the right time in your component.

COMMON USE CASES

ngOnInit()	Fetch initial data, set default values.
ngOnChanges()	React to input property changes.
ngDoCheck()	Custom change detection logic.
ngAfterViewInit()	Access view elements or child components.
ngOnDestroy()	Cleanup, unsubscribe, release resources.

BEST PRACTICES

- Keep hooks lightweight and focused.
- Avoid heavy logic in ngDoCheck().
- Always unsubscribe in ngOnDestroy().
- Prefer services for business logic.
- Understand the sequence to avoid timing issues.

Understanding the component lifecycle helps you write predictable, efficient, and maintainable Angular applications.

Lifecycle = Control + Performance + Reliability

`ngOnInit`

The hook nearly every component with real logic implements -- the standard place to initialize.

✅ Runs Once, After Inputs Set

Unlike the constructor, ngOnInit fires after Angular has assigned the component's @Input values.

📡 Kick Off Initial Fetches

The idiomatic place to call this.customerService.getCustomer(id) for a detail page.

🚫 Not for Every Component

A purely presentational component (like CustomerCard) often needs no ngOnInit at all.

KEY TAKEAWAY ngOnInit is where a component does its initial setup work, once its inputs are guaranteed to be available -- implement OnInit and define the method.

`ngOnInit`

What is `ngOnInit`?

`ngOnInit` is an Angular lifecycle hook that is called **once** after the component's input properties have **been initialized.**

When It Runs

After Angular has set all input-bound properties and before the component's view is rendered.

Primary Use

Ideal for initialization logic such as fetching data, setting up subscriptions, or preparing component state.

Runs Once

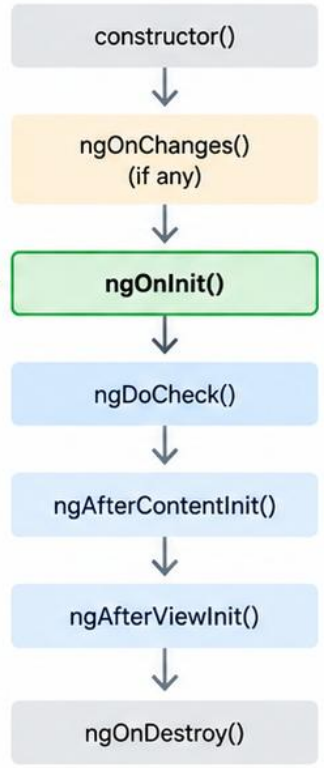
`ngOnInit` is called only once during the component's lifecycle, even if input properties change later.

Avoid

Do not put logic that depends on the view here. Use `ngAfterViewInit` for view-related tasks.

```typescript
import { Component, Input, OnInit } from '@angular/core';

@Component({
  selector: 'app-user-list',
  templateUrl: './user-list.component.html',
  styleUrls: ['./user-list.component.css']
})
export class UserListComponent implements OnInit {

  @Input() organizationId!: number;

  users: any[] = [];

  ngOnInit(): void {
    // Initialization logic
    this.loadUsers();
  }

  loadUsers(): void {
    // Call service to fetch data
  }
}
```

Angular Component Lifecycle



Key Takeaway: Use `ngOnInit` for component initialization that depends on input properties or services.

COMPONENT DESTRUCTION

ngOnDestroy is the last chance to clean up before Angular removes a component for good.

ngOnDestroy Hook

Implement OnDestroy and define ngOnDestroy() to run cleanup logic exactly once, right before removal.

Unsubscribe Manually

Any RxJS .subscribe() not managed automatically must be unsubscribed here to avoid a memory leak.

Clear Timers & Listeners

setInterval/setTimeout handles and manually added DOM listeners must be cleared here too.

KEY TAKEAWAY A component that opens a subscription, timer, or listener owns the responsibility to close it in ngOnDestroy -- Module 34 covers safer alternatives.

COMPONENT DESTRUCTION

The final phase of the Angular component lifecycle. Angular removes the component from the DOM, cleans up resources, and prevents memory leaks.

ngOnDestroy()

Angular calls this hook just before the component is destroyed.
Use it to release resources, unsubscribe from observables, and perform cleanup logic.

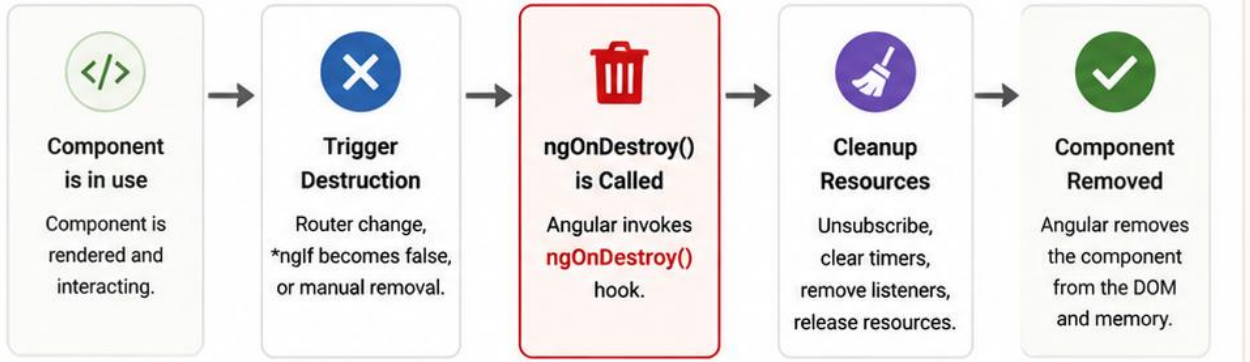
Cleanup Responsibilities

- Unsubscribe from Observables and Subjects
- Clear timers (setTimeout / setInterval)
- Remove event listeners
- Release large objects / references
- Close WebSocket connections

Why It Matters

- Prevents memory leaks
- Avoids unintended side effects
- Ensures better application performance
- Keeps long-running apps stable

COMPONENT DESTRUCTION FLOW



EXAMPLE: USING ngOnDestroy()

```
export class EmployeeListComponent implements OnDestroy {
  private subscription = new Subscription();

  ngOnInit() {
    this.subscription = this.employeeService.getEmployees()
      .subscribe(data => this.employees = data);
  }

  ngOnDestroy(): void {
    this.subscription.unsubscribe();   // Prevent memory leak
    console.log('Component destroyed and cleaned up');
  }
}
```

TIPS

- Always implement ngOnDestroy() when your component subscribes, listens, or allocates resources.
- Unsubscribe even if the observable completes—good habit for maintainability.
- Use takeUntilDestroyed() (Angular 16+) for a cleaner approach.

A well-cleaned component is a well-behaved component. | Lifecycle = Control + Performance + Reliability

TRY IT YOURSELF — EXERCISE 4: PLAN LIFECYCLE AND CLEANUP

Reinforces: standalone components, the lifecycle overview, ngOnInit, and component destruction you just covered.

STEPS (notes/lab33-lifecycle.md)

Step	What To Do
1 — Standalone Imports	Say what a standalone component must declare to use CommonModule directives.
2 — Init Work	Name what belongs in ngOnInit and why it is not in the constructor.
3 — Destroy Work	Name one thing that must be released when the component is destroyed.
4 — Leak Symptom	Describe what a user would notice if that cleanup were skipped.

Lifecycle placement

```
constructor : inject dependencies only -- no data loading
ngOnInit    : initial load, read inputs (they are set by now)
ngOnDestroy : unsubscribe, clear timers -- or the handler outlives the view
```

TRY IT NOW — Complete Exercise 4 before continuing: exercises/exercise-04-lifecycle-and-cleanup.md (workspace: examples/module-33-exercises).

UI LAYERING

Not every component belongs at the same conceptual level -- pages, features, and shared pieces differ.

Page / Routing Layer

Components mapped directly to a route -- CustomerListPage, CustomerDetailPage. Mostly layout.

Feature Layer

Components specific to one feature area -- CustomerCard, CustomerFilterBar -- reused within that feature.

Shared Layer

Generic, feature-agnostic components -- Button, Modal, LoadingSpinner -- reused across the whole app.

KEY TAKEAWAY Draw a mental line between page, feature, and shared components -- it predicts both where new code belongs and what should never import what.

UI LAYERING

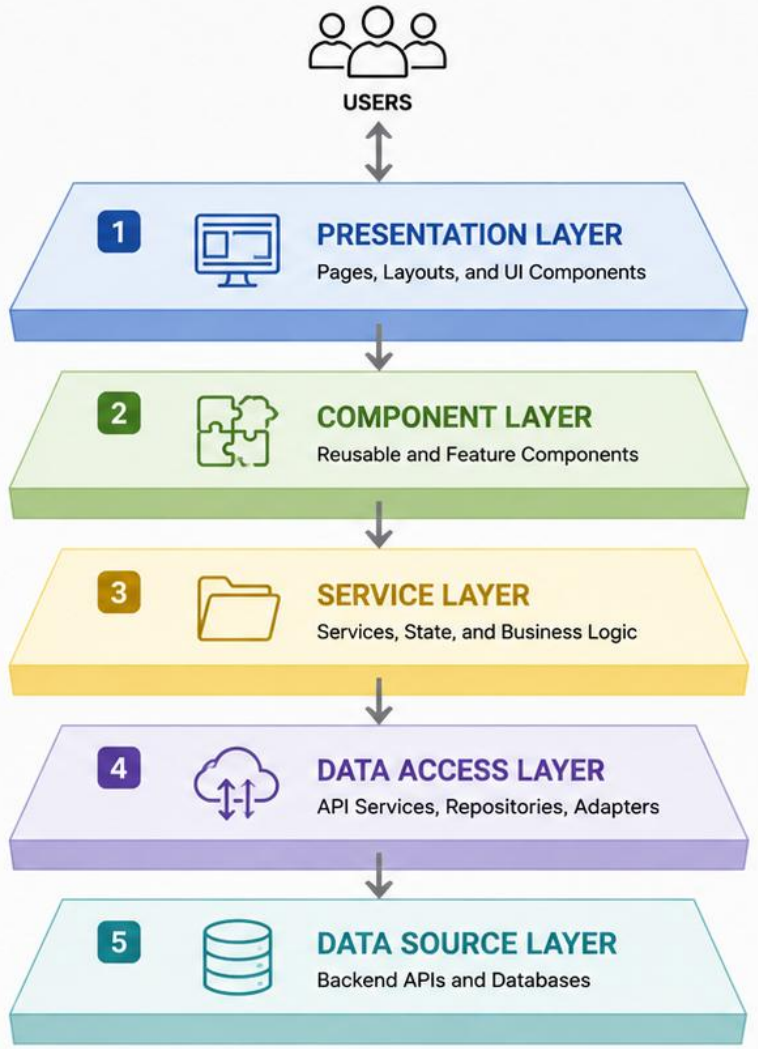
UI layering organizes the user interface into separate layers based on responsibility. This improves maintainability, scalability, reusability, and testability.

KEY PRINCIPLES

- Separate concerns across layers
- Communication flows in one direction
- Each layer has a well-defined responsibility
- Reuse UI components across features
- Keep business logic out of the UI
- Make components small and focused

BENEFITS

- Easier to maintain and refactor
- Better separation of concerns
- Improved reusability of components
- Consistent look and feel
- Faster development
- Easier testing at each layer



RESPONSIBILITY	EXAMPLES
Handles overall page structure and layout composition. Hosts router outlets.	• AppComponent • Feature Pages • Layout Components
Displays UI and captures user input. Stateless or state-aware components.	• Smart Components • Presentational Components • Shared UI Components
Contains business logic, state management, and coordinates data.	• Angular Services • State Services • Facade Services
Handles communication with backend systems and data transformation.	• HttpClient Services • Repositories • Data Adapters
External data providers such as REST APIs, Databases, or Microservices.	• REST API • PostgreSQL • External Services

Layering Guideline: Build small, focused components. Keep business logic in services. Communicate through well-defined interfaces.

SMART VS PRESENTATIONAL COMPONENTS

A widely used pattern: separate components that know things from components that only display things.

Aspect	Smart (Container)	Presentational (Dumb)
Owns data fetching?	Yes -- injects services, calls APIs	No -- receives everything via @Input
Has business logic?	Yes -- decides what data to show	No -- only formats what it is given
Reusable across features?	Rarely -- tied to one use case	Highly -- no knowledge of data source
Example in crm-ui	CustomerListPageComponent	CustomerCardComponent
Testing style	Integration-style, may mock services	Pure unit tests -- pass inputs, assert output

KEY TAKEAWAY Smart components fetch and decide; presentational components receive and display -- keeping them separate maximizes how much of your UI is easily testable and reusable.

SMART VS PRESENTATIONAL COMPONENTS

Angular applications are more maintainable and scalable when we separate container (smart) components that handle logic from presentational (dumb) components that focus on UI and presentation.

SMART (CONTAINER) COMPONENTS

Handle data, logic, and application state

- Connect to services and fetch data
- Contain business logic and state management
- Subscribe to Observables / Signals
- Pass data to child components via @Input()
- Handle events from child components via @Output()
- Decide what to display and when
- Rarely contain HTML markup (delegates to children)
- Example: EmployeeListContainerComponent

VS

PRESENTATIONAL (DUMB) COMPONENTS

Focus on UI, display, and user interaction

- Receive data through @Input()
- Display data and UI elements
- Emit user events through @Output()
- No direct dependency on services
- No business logic or data fetching
- Reusable and easy to test
- Example: EmployeeTableComponent

KEY BENEFITS

- Clear separation of concerns
- Easier to test and maintain
- Reusable UI components
- Scalable application architecture

FLOW

Smart components own the data and logic.
Presentational components focus on how things look.

EXAMPLES

EmployeeListContainerComponent	→	EmployeeTableComponent
EmployeeFormContainerComponent	→	EmployeeFormComponent
DashboardContainerComponent	→	DashboardCardComponent

BEST PRACTICE: Keep components focused. Put logic in smart components and UI in presentational components.

REUSABLE COMPONENT DESIGN

A few concrete habits turn a one-off component into one your whole team reaches for by default.

Accept Data, Not Assumptions

Take a plain Customer via @Input, not a hardcoded API call -- let the caller decide the data source.

Configurable, Not Hardcoded

An @Input for size or variant (e.g. compact vs full CustomerCard) beats copy-pasting a near-identical component.

Emit Intent, Not Implementation

Emit a 'selected' event with the Customer -- let the parent decide what selecting actually does.

KEY TAKEAWAY Reusability is a design habit, not a framework feature -- accept data through inputs, expose configuration through more inputs, and emit intent through outputs.

REUSABLE COMPONENT DESIGN

Reusable components are generic, configurable, and self-contained UI building blocks that can be used across multiple features and screens.

DESIGN PRINCIPLES

- Single Responsibility – Do one thing well
- Configurable – Use Inputs for flexibility
- Composable – Nest and reuse easily
- Isolated – No direct dependency on services
- Accessible – Follow accessibility guidelines
- Testable – Easy to unit test in isolation
- Consistent – Follow design system and standards

BEST PRACTICES

- Keep components small and focused
- Expose configuration via @Input() properties
- Emit events via @Output()
- Use content projection for flexible layouts
- Avoid hard-coded text, styles, and data
- Document component API and usage
- Use trackBy with *ngFor for performance

EXAMPLE: REUSABLE BUTTON COMPONENT

Primary	Secondary	Outline	Icon
Save	Cancel	Learn More	♡

COMPONENT API

@Input() Properties

Property	Type	Purpose
label	string	Button text
type	'button' | 'submit' | 'reset'	HTML button type
variant	'primary' | 'secondary' | 'outline'	Visual style variant
disabled	boolean	Disable the button
icon	string	Icon name (optional)

@Output() Events

Event	Type	Purpose
clicked	EventEmitter<void>	Emitted when button is clicked

ANGULAR EXAMPLE

button.component.ts

```ts
@Component({
  selector: 'app-button',
  templateUrl: './button.component.html',
  styleUrls: ['./button.component.scss']
})
export class ButtonComponent {
  @Input() label: string = 'Button';
  @Input() type: 'button' | 'submit' | 'reset' = 'button';
  @Input() variant: 'primary' | 'secondary' | 'outline' = 'primary';
  @Input() disabled = false;
  @Input() icon?: string;
  @Output() clicked = new EventEmitter<void>();

  onClick() {
    if (!this.disabled) {
      this.clicked.emit();
    }
  }
}
```

button.component.html

```html
<button [type]="type"
        [ngClass]="variant"
        [disabled]="disabled"
        (click)="onClick()"
        class="btn">
  <i *ngIf="icon" class="icon" [class]="icon"></i>
  <ng-content>{{ label }}</ng-content>
</button>
```

WHERE TO USE

Buttons · Cards · Tables · Forms · Modals · Alerts · Badges · Dropdowns · And more...

Reusable components accelerate development, ensure consistency, and make applications easier to maintain. | **Build once. Use everywhere.**

FEATURE-BASED FOLDER STRUCTURE

Organize by what a component is about, not by what kind of file it is.

Folder	Contains
src/app/features/customers/	Everything customer-related: list, detail, card, and customer.service.ts.
src/app/features/invoices/	Everything invoice-related, structured the same way, fully independent of customers/.
src/app/shared/	Feature-agnostic components, pipes, and directives used across multiple features.
src/app/core/	App-wide singletons: auth guards, HTTP interceptors, app-level services.
src/app/app.routes.ts	Top-level route table wiring URL paths to each feature's routed components.

KEY TAKEAWAY Group files by feature (customers/, invoices/), not by file type (all components/, all services/) -- it scales far better as an app grows.

FEATURE-BASED FOLDER STRUCTURE

Organize your Angular application by features instead of by technical type. This improves scalability, maintainability, team collaboration, and discoverability.

WHY FEATURE-BASED?

- Encapsulates everything related to a feature
- Easier to understand and navigate
- Supports independent feature development
- Scales well as the application grows
- Aligns with business domains and teams

BEST PRACTICES

- Keep features small and focused
- Each feature owns its components, services, models, and routing
- Use shared for truly common utilities/components
- Use core for singletons, interceptors, and guards
- Lazy load feature modules/routes
- Name features by business capability

EXAMPLE PROJECT STRUCTURE

```
src
  app
    core                          Singletons, guards, interceptors, config, services
    shared                        Reusable components, directives, pipes, utilities
    features
      employee                    Employee feature module
        components                Employee-specific components
        services                  Employee services
        models                    Employee models/interfaces
        pages                     Employee pages/views
        employee-routing.module.ts  Feature routing
        index.ts                  Public API of the feature
      department                  Department feature module
        components
        services
        models
        pages
        department-routing.module.ts
        index.ts
      auth                        Authentication feature module
    app-routing.module.ts         Application routing
    app.component.ts              Root component
    main.ts                       Application bootstrap
```

FEATURE MODULE CONTENT

components/	UI components specific to the feature
services/	Business logic and API services
models/	Interfaces, types, and models
pages/	Smart/page components
*-routing.module.ts	Feature-specific routes
index.ts	Exports for easy imports

EXAMPLE IMPORTS

```
// Importing a service from employee feature
import { EmployeeService } from
   '@/app/features/employee/services/employee.service';

// Importing a component
import { EmployeeListComponent } from
   '@/app/features/employee/components/employee-list/
   employee-list.component';
```

KEY TAKEAWAY: Organize by features, not by file types. Each feature should be self-contained and independent. | **Better Structure. Better Collaboration. Better Apps.**

SHARED COMPONENTS

The feature-agnostic pieces every part of the app can use without pulling in domain knowledge.

ButtonComponent

A styled button accepting variant/size inputs -- no idea what it will be clicked to do.

LoadingSpinnerComponent

A generic loading indicator any feature can show while data is in flight.

ModalComponent

A wrapper using content projection (<ng-content>) so any feature can put its own content inside.

KEY TAKEAWAY A shared component earns its place in shared/ by having zero knowledge of any specific feature's domain -- if it knows what a Customer is, it belongs in features/customers/ instead.

SHARED COMPONENTS

Shared components are reusable UI building blocks used across multiple features and modules in the application.

WHY USE SHARED COMPONENTS?

- Promotes reuse and consistency
- Reduces duplication of code and styles
- Speeds up development
- Centralizes common UI and logic
- Improves maintainability
- Provides ta single source of truth

WHAT TO SHARE?

- UI components (buttons, cards, tables, modals)
- Directives and pipes
- Validators
- Utility services
- Constants and enums
- Models / interfaces used across features

FOLDER STRUCTURE EXAMPLE

src	
app	
core	Singletons, guards, interceptors, configs
shared	Reusable by all features
components	Reusable UI components
directives	Custom directives
pipes	Custom pipes
services	Utility services
models	Shared interfaces/models
constants	Constants and enums
validators	Form validators
styles	Shared styles and themes
features	Feature modules
employee	
department	
app-routing.module.ts	Application routing
app.component.ts	Root component
main.ts	Application bootstrap

COMMON SHARED COMPONENTS EXAMPLES

ButtonComponent	Standardized button with variants (primary, secondary, outline, text)
DataTableComponent	Reusable data table with sorting, pagination, and actions
ModalComponent	Reusable modal for confirmations, forms, and messages
AlertComponent	Success, error, warning, and info alerts
LoadingSpinnerComponent	Global loading indicator
BreadcrumbComponent	Navigation breadcrumb display

EXAMPLE USAGE

```
// Using a shared component in a feature template

<app-alert type="success" message="Employee saved successfully!"></app-alert>

<app-data-table [data]="employees"
  (pageChange)="onPageChange($event)"></app-data-table>

<app-button variant="primary" (clicked)="onCreate()">
   Create Employee
</app-button>
```

BEST PRACTICE: Keep shared components stateless when possible. Place only truly reusable items in the shared folder. | **Reuse Today. Maintain Easily. Scale Confidently.**

LAYOUT COMPONENTS

A distinct kind of shared component whose job is arranging other components, not rendering content.

AppShellComponent

Renders the persistent header, sidebar nav, and a <router-outlet> for whatever page is active.

PageContainerComponent

Standardizes page padding/max-width so every feature's pages share consistent spacing.

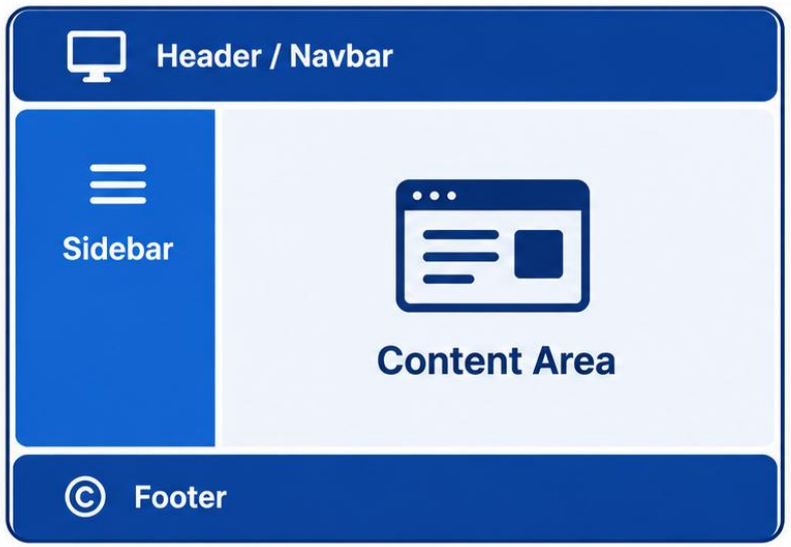
GridComponent

A reusable responsive grid wrapper, driven entirely by content-projected children.

KEY TAKEAWAY Layout components answer 'where does content go,' not 'what content is shown' -- keep that boundary clean and every page inherits consistent structure for free.

Layout Components

Building Consistent and Reusable UI Structure



Header / Navbar
- App branding and navigation
- Present on most pages

Sidebar
- Primary navigation
- Optional or collapsible

Content Area
- Main page content
- Hosts routed views and components

Footer
- Additional links and information
- Consistent across the application

Goal: Use a consistent layout structure to create a clean, reusable, and maintainable Angular UI.

TRY IT YOURSELF — EXERCISE 5: SPLIT SMART AND PRESENTATIONAL

Reinforces: UI layering, the smart-versus-presentational split, reusable design, and feature folders you just covered.

STEPS (notes/lab33-component-map.md)

Step	What To Do
1 — Classify	Mark CustomerListPage and CustomerRow as smart or presentational, with reasons.
2 — Folder Map	Write the folder tree for features/customers and shared.
3 — Reuse Test	Give the test that decides whether a component belongs in shared.
4 — Fixtures	Say where Amina and Ravi come from while there is no API yet.

Component layering

```
smart          CustomerListPage  -- injects services, owns state
presentational CustomerRow       -- @Input in, @Output out, no services
shared/        used unchanged by a second feature
```

TRY IT NOW — Complete Exercise 5 before continuing: exercises/exercise-05-smart-presentational-map.md (workspace: examples/module-33-exercises).

ANGULAR SERVICES OVERVIEW

Shared logic and state that does not belong inside any single component lives in a service.

@Injectable Class

A plain class decorated with @Injectable({ providedIn: 'root' }) -- Angular can construct and share it.

HTTP Calls Live Here

CustomerService wraps HttpClient calls to the CRM backend -- components never call HttpClient directly.

Shared Across Components

One CustomerService instance can be injected into the list page, the detail page, and a search widget alike.

KEY TAKEAWAY A service is where logic that multiple components need to share -- especially HTTP calls -- belongs, keeping components focused purely on presentation and coordination.

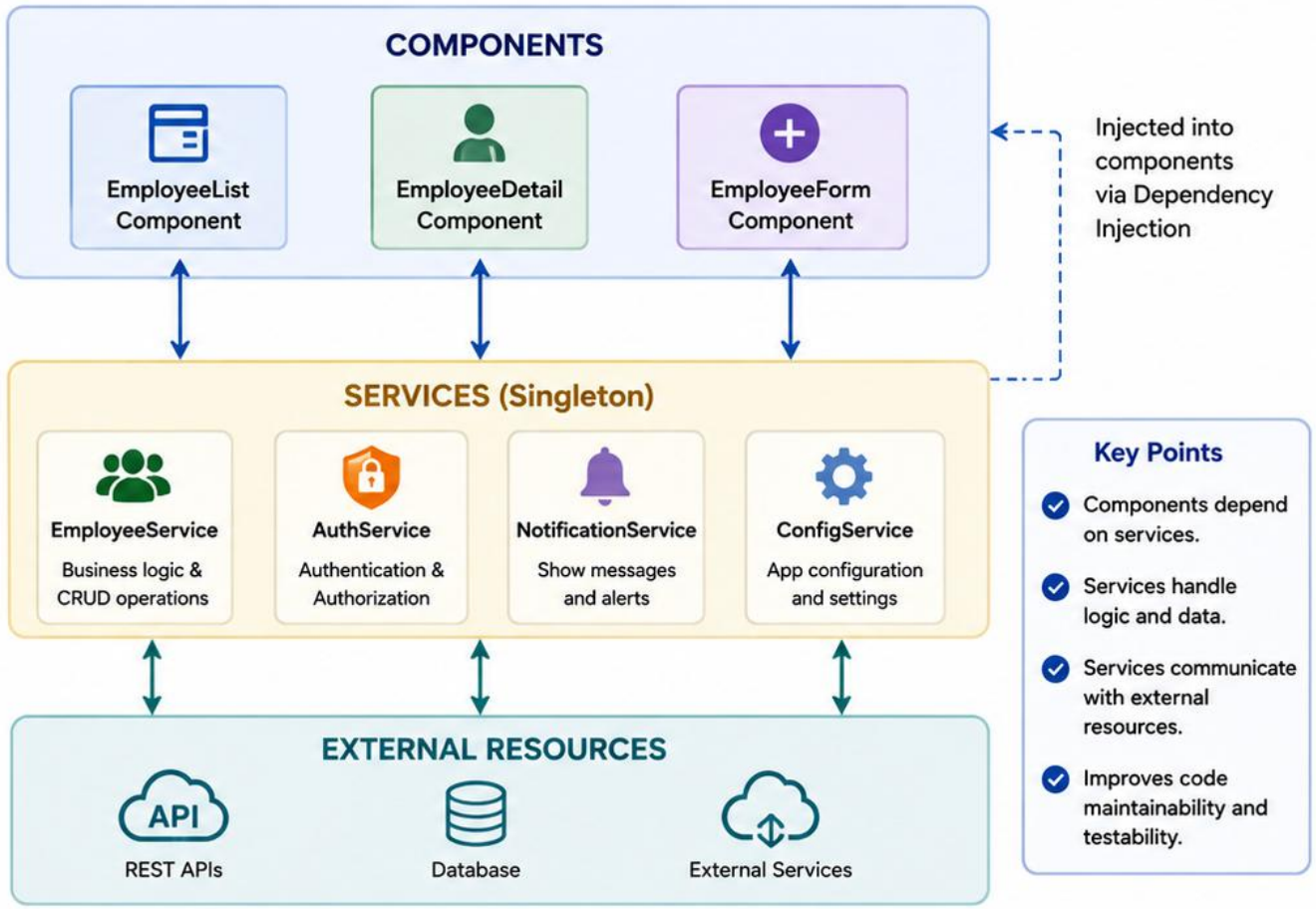
Angular Services Overview

What is an Angular Service?

Services are reusable, singleton objects that encapsulate business logic, data access, and shared functionality across components.

- **Reusability:** Share logic and data across multiple components.
- **Separation of Concerns:** Keep components focused on UI and user interaction.
- **Centralized Data Access:** Handle HTTP requests and interact with backend APIs.
- **Singleton by Default:** A single instance is provided across the application (using Angular DI).
- **Testability:** Easily mockable for unit and integration testing.

How Services Work in Angular



DEPENDENCY INJECTION IN ANGULAR

Angular constructs and supplies a class's dependencies automatically -- you only declare what you need.

providedIn: 'root'

Registers a service with Angular's root injector -- available app-wide as a singleton.

Constructor Injection

constructor(private customerService: CustomerService) {} -- the classic, still-common pattern.

inject() Function

customerService = inject(CustomerService); -- the modern alternative, usable outside constructors too.

KEY TAKEAWAY Whether via constructor parameters or the inject() function, Angular resolves and supplies every dependency by type -- your class never constructs its own dependencies with 'new'.

Dependency Injection in Angular

Angular's DI system provides components and services with their dependencies, making applications modular, testable, and maintainable.

Key Benefits

Loose Coupling
Components depend on abstractions, not concrete implementations.

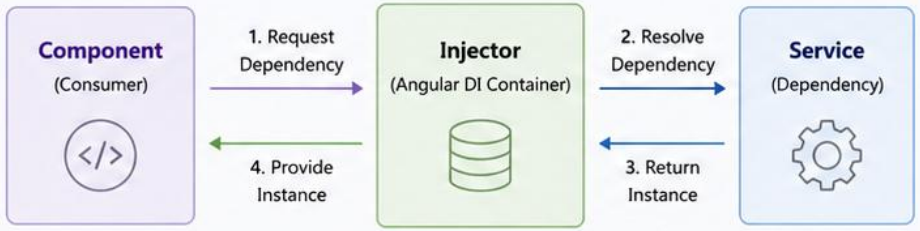
Testability
Easily mock dependencies for unit testing.

Reusability
Services can be shared across multiple components.

Maintainability
Centralized configuration and clear dependency management.

Scalability
DI works seamlessly across large Angular applications.

How Dependency Injection Works



Example

Service

```
@Injectable({
  providedIn: 'root'
})
export class EmployeeService {

  getEmployees() {
    return ['Alice', 'Bob'];
  }
}
```

Component

```
@Component({
  selector: 'app-employee-list',
  templateUrl: './employee-list.component.html'
})
export class EmployeeListComponent {
  employees: string[] = [];

  // Dependency injected via constructor
  constructor(private employeeService: EmployeeService) {}

  ngOnInit() {
    this.employees = this.employeeService.getEmployees();
  }
}
```

How to Provide Services

1. providedIn: 'root' (Recommended)

```
@Injectable({ providedIn: 'root' })
```

Makes the service available application-wide using a tree-shakable provider.

2. Component Providers

```
@Component({
  providers: [MyService]
})
```

Creates a new instance of the service for that component and its children.

3. Module Providers

```
@NgModule({
  providers: [MyService]
})
```

Provides the service to all components declared in the module.

Best Practices

- [x] Use providedIn: 'root' for singleton services.
- [x] Depend on abstractions (interfaces or tokens).
- [x] Keep services focused and single-responsibility.
- [x] Avoid manual instantiation with new.

Angular's Dependency Injection is a powerful pattern that promotes clean architecture, reusability, and testability across your application.

ANGULAR COMPONENT DESIGN PATTERNS

Recurring, named shapes that experienced Angular teams reach for instead of inventing structure ad hoc.

Container/Presentational

Split data-owning components from pure-display components -- covered in depth earlier this module.

Facade Service

One service exposes a simplified API over several underlying services, hiding orchestration complexity.

Composition Over Inheritance

Prefer combining small components/directives over deep class inheritance chains.

Smart Form Component

A dedicated component owns a Reactive Form's FormGroup, validation, and submission -- covered in Module 34.

KEY TAKEAWAY None of these patterns are Angular-specific inventions -- they are general component-design wisdom that Angular's structure makes easy to apply consistently.

Angular Component Design Patterns

Proven patterns to build scalable, maintainable, and testable Angular applications.

Why Use Patterns?

Consistency
Follow tried-and-tested approaches across teams.

Maintainability
Well-structured components are easier to understand and evolve.

Reusability
Design components that are reusable in multiple contexts.

Testability
Patterns encourage separation of concerns and testable code.

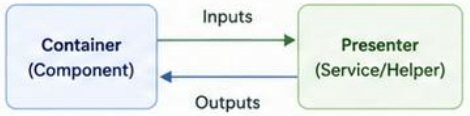
Scalability
Keep your application scalable as it grows.

Key Takeaway
Use design patterns as blueprints, not rigid rules. Adapt them to fit your application needs.

1 Smart / Presentational Pattern

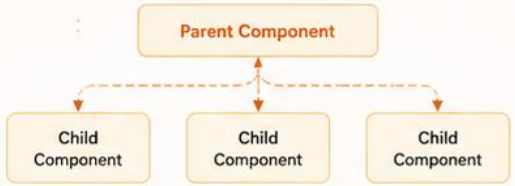
- Smart component handles data and logic.
- Presentational component focuses on UI.
- Improves testability and reusability.

2 Container / Presenter Pattern



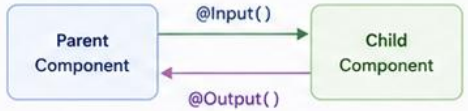
- Moves UI logic to a presenter/service.
- Thin components, rich services.
- Improves separation and testability.

3 Component Composition Pattern



- Build complex UIs by composing small reusable components.
- Enhances maintainability and readability.

4 Input / Output (IO) Pattern



- Share data down using `@Input()`.
- Send events up using `@Output()`.
- Core pattern for parent–child communication.

5 Service Injection Pattern

- Inject services for shared logic and data.
- Promotes reusability and single responsibility.
- Use `providedIn: 'root'` for singleton services.

6 Route-Driven Component Pattern

- UI is driven by routes and lazy-loaded features.
- Improves performance and modularity.
- Ideal for large-scale Angular applications.

Best Practices

- Keep components small and focused.
- Prefer composition over inheritance.
- Encapsulate logic in services.
- Use clear naming and folder structure.
- Test components in isolation.

ENTERPRISE ANGULAR ARCHITECTURE

The patterns from this module, scaled to a codebase with dozens of developers and hundreds of components.

Domain-Driven Feature Folders

Large orgs mirror backend bounded contexts in the front end's features/ folder for the same team boundaries.

Shared Design-System Package

Shared/ components often graduate into a separately versioned, publishable UI library across multiple apps.

Strict Lint Rules on Layering

ESLint rules enforce that shared/ never imports from features/, catching architecture drift automatically.

KEY TAKEAWAY Enterprise Angular architecture is this module's patterns applied consistently and enforced automatically, not a different set of rules for large teams.

Enterprise Angular Architecture

A scalable, maintainable, and modular architecture for large Angular applications.

Architecture Principles

Modularity
Organize by feature and domain with clear boundaries.

Scalability
Structure code to support growth and parallel development.

Reusability
Create shared components, directives, and services.

Maintainability
Follow consistent patterns, naming, and folder structure.

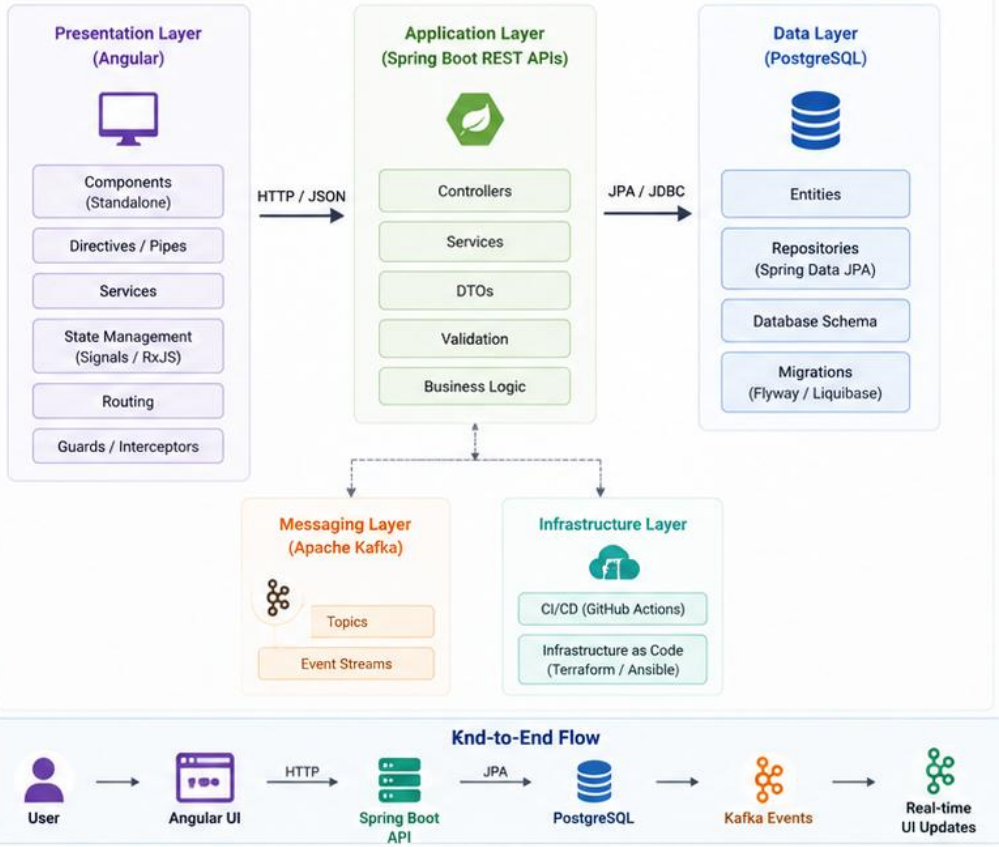
Testability
Design for testability with isolation and dependency injection.

Key Technologies

Angular (Standalone) · TypeScript · RxJS (Observables) · DI (Services)

REST APIs · Spring Boot · PostgreSQL · Kafka

High-Level Architecture



Knd-to-End Flow

Project Structure (Feature-Based)

```
src/app
  core/
    interceptors/
    guards/
    services/
    models/
    constants/
    utils/
  shared/
    components/
    directives/
    pipes/
  features/
    employees/
    departments/
    reports/
    auth/
  layouts/
    header/
    sidebar/
    footer/
  app.routes.ts
  app.config.ts
```

Cross-Cutting Concerns

Security
JWT Authentication, Role-Based Access

Error Handling
Global Error Interceptor, Standardized Responses

Logging & Monitoring
Client Logging, Centralized Monitoring

Performance
Lazy Loading, Code Spiitting, OnPush Change Detection

Accessibility
WCAG Compliance, Keyboard Navigation

Best Practices

- Use Standalone Components and Feature Modules.
- Keep components small and focused.
- Use services for shared logic and communication.
- Prefer reactive patterns (Signals / RxJS).
- Implement lazy loading and preloading strategies.
- Enforce consistent naming and folder structure.
- Write tests for components, services, and APIs.

A well-architected Angular application results in cleaner code, faster delivery, easier maintenance, and long-term enterprise success.

COMMON ANGULAR COMPONENT MISTAKES

The same handful of mistakes account for most avoidable Angular component bugs.

Mistake	Fix
Fetching data inside a presentational component	Move the fetch to a smart/container component or service; pass data via @Input.
Forgetting to unsubscribe an RxJS subscription	Use takeUntilDestroyed(), the async pipe, or prefer signals (Module 34).
Mutating an @Input object directly	Treat inputs as read-only; emit an event and let the parent update its own state.
Skipping 'track' in an @for loop	Always provide a stable track expression so Angular can efficiently diff list updates.
Putting business logic in the constructor	Use ngOnInit for setup that depends on inputs; keep constructors DI-only.

KEY TAKEAWAY Every mistake in this table has a specific, well-known fix -- treat this table as your pre-commit checklist for any new Angular component.

Common Angular Component Mistakes

Avoid these frequent mistakes to build maintainable, performant, and scalable Angular applications.

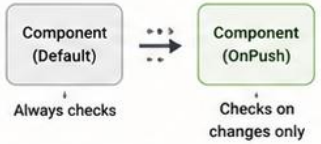
1 Overusing Logic in Templates

```
<div>{{ getFullName(user) }}
  {{ items.length > 0 ?
    'Yes' : 'No' }}
</div>
```

Problem: Makes templates complex, hard to read, and affects performance.

Better: Move logic to the component class and keep templates simple.

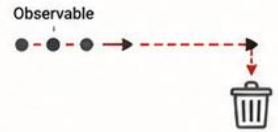
2 Ignoring OnPush Change Detection



Problem: Default strategy can cause unnecessary change detection cycles.

Better: Use OnPush strategy and immutable data patterns.

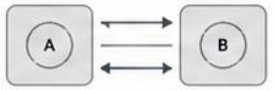
3 Not Unsubscribing from Observables



Problem: Causes memory leaks and unexpected behavior.

Better: Unsubscribe using `takeUntil`, `AsyncPipe`, or `DestroyRef`.

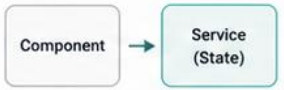
4 Tight Coupling Between Components



Problem: Hard to reuse, test, and maintain.

Better: Communicate via `@Input()`, `@Output()`, and shared services.

5 Using Services for Component-Only State



Problem: Pollutes services and makes state harder to track.

Better: Use component state, Signals, or RxJS for local UI state.

6 Not Using TrackBy in *ngFor

```
<li *ngFor="let item of items">
  {{ item.name }}
</li>
```

Problem: Re-renders all items when list changes, causing performance issues.

Better: Use `trackBy` to improve rendering performance.

```
<li *ngFor="let item of items; trackBy: trackById">
  {{ item.name }}
</li>
```

7 Deep Component Nesting

Problem: Hard to maintain and understand component hierarchy.

Better: Flatten the structure and use smart container components.

8 Not Handling Loading, Error, and Empty States

Problem: Poor user experience when data is loading or fails.

Better: Always handle all 3 states in your UI.

9 Mutating @Input() Data

Problem: Breaks one-way data flow and causes unpredictable bugs.

Better: Treat `@Input()` data as immutable.

10 Ignoring Component Lifecycle

Problem: Leads to incorrect initialization and resource cleanup issues.

Better: Understand and use lifecycle hooks appropriately.

Quick Reminders

- Keep templates simple and declarative.
- Prefer OnPush for better performance.
- Manage subscriptions properly.
- Use Inputs, Outputs, and Services wisely.
- Keep component state in the component.
- Handle loading, error, and empty states.
- Follow Angular best practices and patterns.

Key Takeaway

Well-designed components are simple, focused, and reusable.
Avoid these common mistakes to build robust and scalable Angular applications.

ANGULAR COMPONENT BEST PRACTICES

A closing checklist -- apply these defaults to every component you write in crm-ui and beyond.

Keep Components Small

If a template scrolls past one screen or a class exceeds ~150 lines, look for a component to extract.

Prefer Signal Inputs

Use input()/input.required() over @Input() in new code -- better type safety, reactive by default.

Design for Testability

If a component needs a mocked backend to unit test, its logic likely belongs in a service instead.

Name Things by Role

CustomerListPageComponent vs CustomerCardComponent -- the name should say smart/page or presentational/shared.

KEY TAKEAWAY Small, signal-based, testable, and clearly named -- apply these four defaults to every component and most architecture problems solve themselves.

Angular Component Best Practices

ANGULAR

Build components that are **maintainable**, **reusable**, **testable**, and scalable.

1. Single Responsibility

Keep components focused on one responsibility. Break large components into smaller, purpose-driven pieces.

2. Reusability

Design components to be generic and reusable across features. Use `@Input()` to make them flexible.

3. Clear Inputs & Outputs

Use `@Input()` and `@Output()` with meaningful names.
Keep component APIs small and well-defined.

4. Smart vs Presentational

Keep business logic in smart components or services.
Keep presentational components dumb and UI-focused.

5. Optimize Performance

Use `OnPush` change detection, `trackBy` with `*ngFor`, and avoid expensive operations in templates.

6. Keep Templates Clean

Avoid complex logic in templates. Use component class for logic and keep templates simple and readable.

7. Follow Folder Structure

Organize components by feature. Keep files co-located: component, template, styles, and tests together.

8. Test Your Components

Write unit tests for component logic, inputs, outputs, and user interactions. Aim for high test coverage.

9. Consistent Naming

Use consistent, descriptive names for components, inputs, outputs, methods, and files.

10. Document & Communicate

Document component purpose, inputs, outputs, and usage. Good communication improves team productivity.

Remember: Well-designed components lead to cleaner code, easier onboarding, fewer defects, and a better experience for your users and your team.

Keep APIs Simple

Think Scalability

Build for Maintainability

Design for Reusability

TRY IT YOURSELF — EXERCISE 6: LAB 33 READINESS SELF-CHECK

Pre-lab gate: sweep your plan against the common-mistakes slide, then confirm the five earlier notes files are genuinely complete.

STEPS (notes/lab33-readiness.md)

Step	What To Do
1 — Mistake Sweep	Check your component map against the common-mistakes slide and fix what matches.
2 — Deliverable Recall	List the Lab 33 deliverables from memory, including the evidence screenshots.
3 — Evidence Plan	Name the screenshots you will capture and where they are saved.
4 — Pass Mark	Mark Pass only if all five earlier notes files are complete, not stubbed.

Readiness gate

```
workspace-plan  component-anatomy  io-contract
lifecycle  component-map              -> all five complete?
evidence: browser list + Elements panel, under notes/screenshots/lab-33/
```

TRY IT NOW — Complete Exercise 6 before Lab 33: exercises/exercise-06-lab33-readiness.md (workspace: examples/module-33-exercises).

LAB OVERVIEW — ANGULAR COMPONENT ARCHITECTURE

Scaffold a standalone Angular CRM UI with smart/presentational components and feature folders |

BUSINESS SCENARIO

- Customer list UI is Angular standalone under `features/customers/`. Smart pages own fixtures; presentational children only render and emit. Seed Amina (`CUS-1001`) and Ravi (`CUS-1002`). No React. No SOAP.

LEARNING OBJECTIVES

- Scaffold a standalone Angular app with the CLI Use templates, interpolation, property/event binding Wire `@Input()` / `@Output()` (or `input()` / `output()`) Separate smart vs presentational responsibilities Lay out feature folders ready for later HttpClient labs

WHAT YOU BUILD

- `examples/lab33-crm-ui/` — feature folders, list/detail components, notes | `lab33-crm-ui` lists CUS-1001 / CUS-1002 via presentational children |

KEY TAKEAWAY Build the Northstar CRM **UI shell in Angular: CLI app, feature folders, **smart** pages that own data, and **presentational** children that take inputs and emit outputs.**

Lab Overview – Angular Component Development

Hands-on lab to design, build, and integrate reusable Angular components using best practices.

ANGULAR

LAB SCENARIO

You are building a feature module for an **Employee Management application.** Your goal is to create reusable, well-structured components that follow Angular best practices and support clean UI and maintainability.

FEATURE MODULE STRUCTURE

- employee
  - components
    - employee-list
    - employee-detail
    - employee-form
    - employee-card
    - confirm-dialog
  - services
  - employee.module.ts

Follow a feature-based structure with focused, reusable components.

WHAT YOU WILL BUILD

1. Reusable Components

Create UI components: EmployeeListComponent, EmployeeDetailComponent, EmployeeFormComponent, EmployeeCardComponent, and ConfirmDialogComponent.

2. Input & Output Bindings

Use @Input() to pass data to child components and @Output() with EventEmitter to emit events to parents.

3. Parent–Child Communication

Implement communication between components to handle selections, form submissions, and dialog actions.

4. Component Composition

Compose components to build the Employee feature page (EmployeeList + EmployeeCard + ConfirmDialog).

5. Best Practices

Apply best practices: clean templates, meaningful naming, standalone components (or NgModule), and separation of concerns.

LEARNING GOALS

- Build and structure Angular components using best practices
- Use @Input() and @Output() effectively
- Enable parent–child communication
- Compose reusable components
- Follow a clean and scalable folder structure
- Improve maintainability and reusability of UI components

TOOLS & TECHNOLOGIES

Angular (17+) | TypeScript | RxJS | Angular CLI

DELIVERABLES

- Feature module with well-structured components
- Working parent–child communication
- Functional UI with forms, lists, and dialogs
- Clean code following Angular best practices

LAB TASKS AND DELIVERABLES

Angular Component Architecture — implementation steps, deliverables, and the evaluation rubric.

#	Implementation Step
1	Open starter/README.md.
2	Scaffold/copy into java-bootcamp/examples/lab33-crm-ui.
3	Fill TODOs for model, list item, smart page, app host.
4	Smoke with ng serve; evidence under notes/screenshots/lab-33/.
5	Mark timed-path Pass; finish remaining Steps as homework.
6	lab33-crm-ui under examples/
7	features/customers/ smart + presentational components
8	List shows CUS-1001 Amina Khan and CUS-1002 Ravi Singh
9	At least one input and one output parent↔child flow
10	docs/component-notes.md (smart vs presentational + folder map)
11	Browser + Elements screenshot
12	ng build or serve smoke evidence

EXPECTED DELIVERABLES

- lab33-crm-ui under examples/ features/customers/ smart + presentational components List shows CUS-1001 Amina Khan and CUS-1002 Ravi Singh At least one input and one output parent↔child flow docs/component-notes.md (smart vs presentational + folder map) Browser + Elements screenshot ng build or serve smoke evidence No secrets / node_modules/ / dist/ committed

RUBRIC (100 MARKS)

- Environment 10 · Core Impl 30 · Integration/Config 15 · Failure Handling 15 · Verification 10 · Security/Prod Awareness 10 · Docs/Evidence 10

KEY TAKEAWAY Fixtures CUS-1001 (Amina Khan, ACTIVE) and CUS-1002 (Ravi Singh, PROSPECT) anchor Lab 33; complete pre-lab exercises 1→6 first, then the graded lab.

Lab Tasks and Deliverables

Build a feature module with reusable Angular components following best practices.

ANGULAR

LAB TASKS

1 Project Setup

Create a new Angular application and generate a feature module named `employee`.

- Use Angular CLI
- Follow feature-based structure

2 Component Development

Create the following components inside the employee module: `EmployeeListComponent`, `EmployeeDetailComponent`, `EmployeeFormComponent`, `EmployeeCardComponent`, `ConfirmDialogComponent`.

- Build component class
- Create HTML template
- Add basic styling (optional)

3 Input & Output Bindings

Use `@Input()` to pass data to child components and `@Output()` with `EventEmitter` to emit events to parents.

- Pass employee data to cards
- Handle form submit & cancel events

4 Parent–Child Communication

Implement communication flow between components to handle selections, form submissions, and dialog actions.

- Display selected employee details
- Open/close confirmation dialog

5 Component Composition

Compose components to build the Employee feature page: EmployeeList + EmployeeCard + EmployeeForm + ConfirmDialog.

- Integrate all components
- Build complete UI workflow

6 Best Practices

Apply Angular best practices for structure, naming, and reusability.

- Use standalone components (or NgModule)
- Keep components focused and reusable

DELIVERABLES

- Employee feature module with well-structured components
- Reusable components with Inputs and Outputs
- Working parent–child communication
- Functional UI with forms, lists, and dialogs
- Clean code following Angular best practices
- README with setup and run instructions

ACCEPTANCE CRITERIA

- Application runs without errors
- All components render and work as expected
- Events flow correctly between components
- Code is modular, readable, and reusable
- Follows naming and folder structure conventions

TIPS

- Keep components small and focused.
- Use meaningful names for components, files, and properties.
- Avoid logic in templates—use component class.
- Reuse components where possible.

SUBMISSION

Submit your code repository link and a brief demo of the working application.

MODULE 33 SUMMARY

Angular applications are trees of standalone components, wired by inputs, outputs, and DI.

KEY CONCEPTS COVERED

Standalone Components

The modern default -- each component declares its own template dependencies directly.

Data Binding

Interpolation, property, event, and two-way binding keep template and class in sync.

Inputs Down, Outputs Up

The one rule that keeps a component tree predictable and reusable at any size.

Lifecycle Hooks

ngOnInit for setup, ngOnDestroy for cleanup -- the two hooks used almost everywhere.

Feature-Based Structure

Organize by domain (features/customers/), not by file type, and keep shared/ domain-agnostic.

Dependency Injection

Services, providedIn: 'root', and inject()/constructor injection supply shared logic and state.

MODULE FLOW RECAP

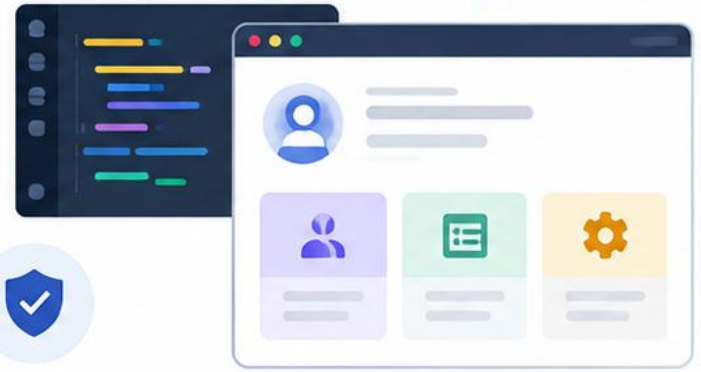
1. CLI & Bootstrap
2. Component Anatomy
3. Binding & I/O
4. Lifecycle
5. Architecture at Scale

KEY TAKEAWAY A component is a class, a template, and a selector; compose small, well-scoped, standalone components with clean input/output contracts and everything else in this module follows.

Module Summary

ANGULAR

You've learned how to design, build, and structure Angular components that are **reusable**, **maintainable**, and **enterprise-ready**.



WHAT YOU LEARNED

- Understand Angular component architecture and structure
- Build components using templates, classes, and metadata
- Use `@Input()` and `@Output()` for component communication
- Implement parent–child communication and component composition
- Organize components with a feature-based folder structure
- Apply best practices for reusable and maintainable code

KEY TAKEAWAYS

- Components are the building blocks of Angular applications.
- Keep components focused, small, and responsible for a single task.
- Use Inputs, Outputs, and EventEmitter for effective communication.
- Compose UIs by combining smaller, reusable components.
- Follow a consistent folder structure and naming convention.
- Leverage standalone components (or NgModules) for scalability.
- Write clean templates and keep business logic in the component class or services.

COMPONENT DEVELOPMENT FLOW

Plan — Define component purpose and API

→ **Build** — Create template, class, and styles

→ **Connect** — Use Inputs, Outputs, and services

→ **Test** — Verify behavior and interactions

→ **Reuse** — Use in multiple places

TOOLS & TECHNOLOGIES

- Angular
- TypeScript
- RxJS
- Angular CLI

You're Ready To:

- Build feature-rich UIs with reusable components
- Scale applications with a clean and maintainable architecture

> *Well-designed components lead to better code, happier teams, and delighted users.*

Next Steps

- Explore advanced component patterns and state management
- Integrate components with services and APIs

KNOWLEDGE CHECK (1 OF 2)

Test your understanding of Angular's structure, components, and templates.

1. What does bootstrapApplication(AppComponent, appConfig) do in main.ts?

A. Declares AppComponent in an NgModule

B. Renders AppComponent into index.html's <app-root>

C. Installs the Angular CLI globally

D. Compiles the TypeScript project to JavaScript

2. Which decorator property lets a component list its own template dependencies without an NgModule?

A. selector

B. providedIn

C. imports

D. exports

3. What is the correct direction of data flow in Angular's parent-child communication pattern?

A. Outputs down, inputs up

B. Inputs down, outputs up

C. Both directions through inputs only

D. Components read parent state directly

4. Which lifecycle hook is the standard place to kick off an initial HTTP fetch?

A. The constructor

B. ngOnDestroy

C. ngOnInit

D. ngAfterViewChecked

KEY TAKEAWAY bootstrapApplication starts a standalone app; a component's own imports replace NgModule; data flows inputs down and outputs up; ngOnInit is where initial fetches belong.

KNOWLEDGE CHECK (2 OF 2)

Test your understanding of component patterns, lifecycle, and Angular architecture.

5. In the Smart vs Presentational pattern, what does a presentational component typically avoid doing?

A. Rendering markup

B. Accepting @Input values

C. Injecting services and fetching its own data

D. Emitting @Output events

6. Where should a manually created RxJS subscription be cleaned up to avoid a memory leak?

A. ngOnInit

B. The constructor

C. ngOnChanges

D. ngOnDestroy

7. In a feature-based structure, where should a domain-agnostic ButtonComponent live?

A. src/app/features/customers/

B. src/app/shared/

C. src/app/core/

D. Directly in src/app/

8. Why is input.required<Customer>() preferred over plain @Input() in new code?

A. It renders faster in every case

B. Compile fails if missing; it integrates with signals

C. It removes the need for a component template

D. It allows two-way binding automatically

KEY TAKEAWAY Presentational components never fetch their own data; subscriptions are cleaned up in ngOnDestroy; shared/ holds domain-agnostic components only; input.required() catches missing bindings at compile time.

Knowledge Check

Test your understanding of **Angular** component architecture and best practices.

Choose the best answer for each question.

1 **What is the primary purpose of a component in Angular?**

- (A) To manage state globally
- (B) To define routing
- (C) To encapsulate UI and behavior for a specific task
- (D) To connect directly to the database

2 **Which decorator is used to define an Angular component?**

- (A) @Injectable
- (B) @Component
- (C) @Directive
- (D) @NgModule

3 **Which of the following is used to pass data from parent to child component?**

- (A) @Output()
- (B) @Input()
- (C) EventEmitter
- (D) @ViewChild()

4 **Which of the following is used by a child component to send data to its parent?**

- (A) @Input()
- (B) @Output() with EventEmitter
- (C) @ViewChild()
- (D) @ContentChild()

5 **What is the benefit of using standalone components?**

- (A) They require NgModule to work
- (B) They reduce boilerplate and improve reuse
- (C) They can only be used in lazy-loaded modules
- (D) They cannot have Inputs or Outputs

6 **Which lifecycle hook is called after Angular sets input properties?**

- (A) ngOnInit()
- (B) ngAfterViewInit()
- (C) ngOnChanges()
- (D) ngOnDestroy()

7 **Which command is used to generate a new component using Angular CLI?**

- (A) ng new component my-comp
- (B) ng g component my-comp
- (C) ng create comp my-comp
- (D) ng generate comp my-comp

8 **What is a best practice for Angular templates?**

- (A) Keep business logic in templates
- (B) Use complex expressions in templates
- (C) Keep templates simple and readable
- (D) Access services directly in templates

9 **How can you share components across multiple feature modules?**

- (A) Declare them in each module
- (B) Use a shared module or standalone components
- (C) Store them in the root component
- (D) Use @Injectable decorator

CHALLENGE QUESTION

You have a component that needs to display a list of employees and allow the user to select one. Which combination should you use for the best implementation?

- (A) Service with global variables
- (B) Parent component with @Input() and child with @Output()
- (C) Direct DOM manipulation
- (D) Event binding on the root component

SELF CHECK

How confident are you with Angular component development?

Not Confident | Somewhat Confident | Neutral | Confident | Very Confident

KEY TAKEAWAYS

- Components are the building blocks of Angular applications.
- Use @Input(), @Output(), and EventEmitter for communication.
- Follow best practices for clean, reusable, and maintainable code.
- Well-designed components lead to scalable and testable applications.

"A component is not a file -- it is a contract between a template and the class behind it."

MODULE 33 COMPLETE

Angular Component Architecture

You can now design, build, and organize standalone Angular components the way production teams do.